Physlib Monthly Report

1–30 November 2025

Contributors: i-love-lean, Joseph Tooby-Smith, Nicola Bernini, Pietro Monticone, Shlok Vaibhav Singh

Reviewers: Joseph Tooby-Smith

Generated 14 August 2026 from commit `b47abf9`.

Abstract

Physlib is a community library of formalised physics for the Lean 4 proof assistant: definitions and results from across physics, stated in a formal language and checked by machine. This report is an automatically generated record of one month's additions to it.

It covers 1–30 November 2025 on branch `master` of `leanprover-community/physlib`, between commits `c6ebae9` and `b47abf9`. Over that period 5 contributors landed 23 commits, changing 19,313 lines across 134 files and adding 478 new Lean declarations, each catalogued with its statement in Section 2. The complete diff is available at GitHub compare.

Contents

1 Introduction

During Nov 2025, 5 contributors submitted 23 commits that changed 19,313 lines (+12,840 / -6,473) across 134 files spanning 21 top-level areas of the repository. Of those, 22 files were newly added and 10 were removed outright. This report catalogues 478 newly added Lean declarations: 392 lemmas, 45 defs, 40 instances, and 1 abbrev. We're delighted to recognize the following first-time contributors this month: i-love-lean and Nicola Bernini.

See the full compare diff and the list of commits on GitHub for the raw source.

1.1 Contributors

The following individuals authored commits merged during this reporting period, listed alphabetically.

Contributor	Lines Changed	Commits
i-love-lean	18	1
Joseph Tooby-Smith	21,057	18
Nicola Bernini	70	1
Pietro Monticone	42	2
Shlok Vaibhav Singh	162	1

1.2 Reviewers

The following individuals approved pull requests during this reporting period, listed alphabetically.

Reviewer	PRs Reviewed
Joseph Tooby-Smith	4

1.3 Modified subfolders

Subfolder	Files	New	Deleted	Additions	Deletions
`PhysLean/SpaceAndTime`	30	+16	-4	+8,152	-2,573
`PhysLean/Electromagnetism`	24	+2	-3	+2,157	-1,656
`PhysLean/QFT`	9	–	–	+1,247	-648
`PhysLean/Mathematics`	13	–	-2	+303	-761
`PhysLean/ClassicalMechanics`	13	+3	-1	+347	-465
`PhysLean/Relativity`	17	–	–	+411	-67
`PhysLean/Particles`	6	–	–	+62	-144
`PhysLean/Optics`	1	–	–	+4	-97
`PhysLean/QuantumMechanics`	4	+1	–	+75	-4
`PhysLean/Cosmology`	1	–	–	+25	-12
`PhysLean.lean`	1	–	–	+24	-12
`lake-manifest.json`	1	–	–	+16	-16
`scripts/MetaPrograms`	4	–	–	+8	-7
`PhysLean/StringTheory`	2	–	–	+2	-2
`lakefile.toml`	1	–	–	+2	-2
`PhysLean/Meta`	1	–	–	+2	-1
`PhysLean/Units`	2	–	–	+0	-2

README.md	1	–	–	+1	-1
lean-toolchain	1	–	–	+1	-1
scripts	1	–	–	+1	-1
PhysLean/Thermodynamics	1	–	–	+0	-1

1.4 New declarations by kind

Kind	Count
lemma	392
def	45
instance	40
abbrev	1

2 Details by file

2.1 PhysLean.Electromagnetism.Kinematics.ElectricField

PhysLean/Electromagnetism/Kinematics/ElectricField.lean on GitHub.

lemma electricField_apply_differentiable [source]

```
lemma electricField_apply_differentiable {A : ElectromagneticPotential d}
    {c : SpeedOfLight}
    (hA : ContDiff ℝ 2 A) :
    Differentiable ℝ (fun (tx : Time × Space d) => A.electricField c tx.1 tx.2 i)
```

lemma electricField_apply_differentiable_time [source]

```
lemma electricField_apply_differentiable_time {A : ElectromagneticPotential d}
    {c : SpeedOfLight}
    (hA : ContDiff ℝ 2 A) (x : Space d) (i : Fin d) :
    Differentiable ℝ (fun t => A.electricField c t x i)
```

def electricField [source]

The electric field of an electromagnetic potential which is a distribution.

```
noncomputable def electricField {d} (c : SpeedOfLight) :
    DistElectromagneticPotential d →ₗ[ℝ]
    (Time × Space d) →d[ℝ] EuclideanSpace ℝ (Fin d) where
```

2.2 PhysLean.Electromagnetism.Kinematics.EMPotential

PhysLean/Electromagnetism/Kinematics/EMPotential.lean on GitHub.

abbrev DistElectromagneticPotential [source]

The electromagnetic potential as a distribution and as a tensor `A^μ`.

```
noncomputable abbrev DistElectromagneticPotential (d : ℕ := 3)
```

def deriv [source]

The derivative of a electromagnetic potential, which is a distribution.

```
noncomputable def deriv {d} : DistElectromagneticPotential d →ₗ[ℝ]
    (SpaceTime d) →d[ℝ] Lorentz.CoVector d ⊗[ℝ] Lorentz.Vector d where
```

lemma deriv_basis_repr_apply [source]

```
@[simp]
lemma deriv_basis_repr_apply {d} {μν : (Fin 1 ⊕ Fin d) × (Fin 1 ⊕ Fin d)}
    (A : DistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ)) :
    (Lorentz.CoVector.basis.tensorProduct Lorentz.Vector.basis).repr (deriv A ε) μν =
    distDeriv μν.1 A ε μν.2
```

lemma toTensor_deriv_basis_repr_apply [source]

```
lemma toTensor_deriv_basis_repr_apply {d} (A : DistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ)) (b : ComponentIdx (S := realLorentzTensor d)
      (Fin.append ![Color.down] ![Color.up])) :
    (Tensor.basis _).repr (Tensorial.toTensor (deriv A ε)) b =
    distDeriv (finSumFinEquiv.symm (b 0)) A ε (finSumFinEquiv.symm (b 1))
```

lemma deriv_equivariant [source]

```
@[sorryful]
lemma deriv_equivariant {d} {A : DistElectromagneticPotential d}
    (Λ : LorentzGroup d) : deriv (Λ • A) = Λ • deriv A
```

2.3 PhysLean.Electromagnetism.Kinematics.FieldStrength

PhysLean/Electromagnetism/Kinematics/FieldStrength.lean on GitHub.

def fieldStrengthAux [source]

An auxillary definition for the field strength of an electromagnetic potential based on a distribution. On Schwartz maps this has the same value as the field strength tensor, but no linearity or continous properties built in.

```
noncomputable def fieldStrengthAux {d} (A : DistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ)) : Lorentz.Vector d ⊗[ℝ] Lorentz.Vector d
```

lemma fieldStrengthAux_eq_add [source]

```
lemma fieldStrengthAux_eq_add {d} (A : DistElectromagneticPotential d) (ε : 𝓢(SpaceTime d, ℝ))
    :
    fieldStrengthAux A ε =
    Tensorial.toTensor.symm (permT id (PermCond.auto) {(η d | μ μ' ⊗ A.deriv ε | μ' ν)}ᵀ)
    - Tensorial.toTensor.symm (permT ![1, 0] (PermCond.auto)
      {(η d | μ μ' ⊗ A.deriv ε | μ' ν)}ᵀ)
```

lemma toTensor_fieldStrengthAux [source]

```
lemma toTensor_fieldStrengthAux {d} (A : DistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ)) :
    Tensorial.toTensor (fieldStrengthAux A ε) =
    (permT id (PermCond.auto) {(η d | μ μ' ⊗ A.deriv ε | μ' ν)}ᵀ)
    - (permT ![1, 0] (PermCond.auto) {(η d | μ μ' ⊗ A.deriv ε | μ' ν)}ᵀ)
```

lemma toTensor_fieldStrengthAux_basis_repr [source]

```
lemma toTensor_fieldStrengthAux_basis_repr {d} (A : DistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ))
    (b : ComponentIdx (S := realLorentzTensor d) (Fin.append ![Color.up] ![Color.up])) :
    (Tensor.basis _).repr (Tensorial.toTensor (fieldStrengthAux A ε)) b =
    ∑ κ, (η (finSumFinEquiv.symm (b 0)) κ * SpaceTime.distDeriv κ A ε (finSumFinEquiv.symm (b
    1)) -
      η (finSumFinEquiv.symm (b 1)) κ * SpaceTime.distDeriv κ A ε (finSumFinEquiv.symm (b 0)))
```

lemma fieldStrengthAux_tensor_basis_eq_basis [source]

```
lemma fieldStrengthAux_tensor_basis_eq_basis {d} (A : DistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ))
    (b : ComponentIdx (S := realLorentzTensor d) (Fin.append ![Color.up] ![Color.up])) :
    (Tensor.basis _).repr (Tensorial.toTensor (A.fieldStrengthAux ε)) b =
    (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr (A.fieldStrengthAux ε)
      (finSumFinEquiv.symm (b 0), finSumFinEquiv.symm (b 1))
```

lemma fieldStrengthAux_basis_repr_apply [source]

```
lemma fieldStrengthAux_basis_repr_apply {d} {μν : (Fin 1 ⊕ Fin d) × (Fin 1 ⊕ Fin d)}
    (A : DistElectromagneticPotential d) (ε : 𝓢(SpaceTime d, ℝ)) :
    (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr (A.fieldStrengthAux ε) μν =
    ∑ κ, ((η μν.1 κ * distDeriv κ A ε μν.2) - η μν.2 κ * distDeriv κ A ε μν.1)
```

lemma fieldStrengthAux_basis_repr_apply_eq_single [source]

```
lemma fieldStrengthAux_basis_repr_apply_eq_single {d} {μν : (Fin 1 ⊕ Fin d) × (Fin 1 ⊕ Fin d)}
    (A : DistElectromagneticPotential d) (ε : 𝓢(SpaceTime d, ℝ)) :
    (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr (A.fieldStrengthAux ε) μν =
    ((η μν.1 μν.1 * distDeriv μν.1 A ε μν.2) - η μν.2 μν.2 * distDeriv μν.2 A ε μν.1)
```

lemma fieldStrengthAux_eq_basis [source]

```
lemma fieldStrengthAux_eq_basis {d} (A : DistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ)) :
    (A.fieldStrengthAux ε) = ∑ μ, ∑ ν,
      ((η μ μ * distDeriv μ A ε ν) - η ν ν * distDeriv ν A ε μ)
      • Lorentz.Vector.basis μ ⊗ₜ[ℝ] Lorentz.Vector.basis ν
```

def fieldStrength [source]

The field strength of an electromagnetic potential which is a distribution.

```
noncomputable def fieldStrength {d} :
    DistElectromagneticPotential d →ₗ[ℝ]
    (SpaceTime d) →d[ℝ] Lorentz.Vector d ⊗[ℝ] Lorentz.Vector d where
```

lemma fieldStrength_eq_basis [source]

```
lemma fieldStrength_eq_basis {d} (A : DistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ)) :
    A.fieldStrength ε = ∑ μ, ∑ ν,
      ((η μ μ * distDeriv μ A ε ν) - η ν ν * distDeriv ν A ε μ)
      • Lorentz.Vector.basis μ ⊗ₜ[ℝ] Lorentz.Vector.basis ν
```

2.4 PhysLean.Electromagnetism.Kinematics.MagneticField

PhysLean/Electromagnetism/Kinematics/MagneticField.lean on GitHub.

def magneticFieldMatrix [source]

The magnetic field matrix of an electromagnetic potential which is a distribution.

```
noncomputable def magneticFieldMatrix {d} (c : SpeedOfLight) :
    DistElectromagneticPotential d →ₗ[ℝ]
    (Time × Space d) →d[ℝ] (EuclideanSpace ℝ (Fin d) ⊗[ℝ] EuclideanSpace ℝ (Fin d)) where
```

lemma magneticFieldMatrix_eq_vectorPotential [source]

```
lemma magneticFieldMatrix_eq_vectorPotential {c : SpeedOfLight}
    (A : DistElectromagneticPotential d)
    (ε : 𝓢(Time × Space d, ℝ)) :
    A.magneticFieldMatrix c ε = ∑ i, ∑ j,
    (Space.distSpaceDeriv j (A.vectorPotential c) ε i -
      Space.distSpaceDeriv i (A.vectorPotential c) ε j) •
    EuclideanSpace.basisFun (Fin d) ℝ i ⊗ₜ[ℝ] EuclideanSpace.basisFun (Fin d) ℝ j
```

2.5 PhysLean.Electromagnetism.Kinematics.ScalarPotential

PhysLean/Electromagnetism/Kinematics/ScalarPotential.lean on GitHub.

def scalarPotential [source]

The scalar potential of an electromagnetic potential which is a distribution.

```
noncomputable def scalarPotential {d} (c : SpeedOfLight) :
    DistElectromagneticPotential d →ₗ[ℝ]
    (Time × Space d) →d[ℝ] ℝ where
```

2.6 PhysLean.Electromagnetism.Kinematics.VectorPotential

PhysLean/Electromagnetism/Kinematics/VectorPotential.lean on GitHub.

def vectorPotential [source]

The vector potential of an electromagnetic potential which is a distribution.

```
noncomputable def vectorPotential {d} (c : SpeedOfLight) :
    DistElectromagneticPotential d →ₗ[ℝ]
    (Time × Space d) →d[ℝ] EuclideanSpace ℝ (Fin d) where
```

2.7 PhysLean.Electromagnetism.PointParticle.FiniteCollection

PhysLean/Electromagnetism/PointParticle/FiniteCollection.lean on GitHub.

lemma electricField_eq_neg_distGrad_electricPotential [source]

```
lemma electricField_eq_neg_distGrad_electricPotential {n : ℕ} (q : Fin n → ℝ) (ε : ℝ)
    (r₀ : Fin n → Space) :
    electricField q ε r₀ = - Space.distGrad (electricPotential q ε r₀)
```

2.8 PhysLean.Electromagnetism.PointParticle.OneDimension

PhysLean/Electromagnetism/PointParticle/OneDimension.lean on GitHub.

def oneDimPointParticle [source]

The electromagnetic potential of a point particle stationary at the origin of 1d space.

```
noncomputable def DistElectromagneticPotential.oneDimPointParticle (q : ℝ) :
    DistElectromagneticPotential 1
```

lemma oneDimPointParticle_vectorPotential [source]

```
@[simp]
lemma DistElectromagneticPotential.oneDimPointParticle_vectorPotential(q : ℝ) :
    (DistElectromagneticPotential.oneDimPointParticle q).vectorPotential 1 = 0
```

lemma oneDimPointParticle_scalarPotential [source]

```
lemma DistElectromagneticPotential.oneDimPointParticle_scalarPotential (q : ℝ) :
    (DistElectromagneticPotential.oneDimPointParticle q).scalarPotential 1 =
    Space.constantTime (distOfFunction (fun x => - (q/(2)) • ‖x‖) (by fun_prop))
```

lemma oneDimPointParticle_electricField [source]

```
lemma DistElectromagneticPotential.oneDimPointParticle_electricField (q : ℝ) :
    (DistElectromagneticPotential.oneDimPointParticle q).electricField 1 =
    (q / 2) • constantTime (distOfFunction (fun x : Space 1 => ‖x‖ ^ (- 1 : ℤ) • basis.repr x)
      (IsDistBounded.zpow_smul_repr_self (- 1 : ℤ) (by omega)))
```

lemma oneDimPointParticle_gaussLaw [source]

```
lemma DistElectromagneticPotential.oneDimPointParticle_gaussLaw (q : ℝ) :
    distSpaceDiv ((DistElectromagneticPotential.oneDimPointParticle q).electricField 1) =
    constantTime (q • diracDelta ℝ 0)
```

2.9 PhysLean.Electromagnetism.PointParticle.ThreeDimension

PhysLean/Electromagnetism/PointParticle/ThreeDimension.lean on GitHub.

lemma distGrad_electricPotential_eq_electricField_of_integral_eq_zero [source]

The relation `E = -∇φ` holds for the point particle if the integral `∫ x, d_y η x * ‖x‖⁻¹ + η x * -⟪(‖x‖ ^ 3)⁻¹ • x, y⟫_ℝ = 0` is zero.

```
lemma distGrad_electricPotential_eq_electricField_of_integral_eq_zero (q ε : ℝ)
    (h_integral : ∀ η : 𝓢(Space 3, ℝ), ∀ y : EuclideanSpace ℝ (Fin 3),
    ∫ (a : Space 3), (fderivCLM ℝ η a (basis.repr.symm y) * ‖a‖⁻¹ +
    η a * - ⟪(‖a‖ ^ 3)⁻¹ • basis.repr a, y⟫_ℝ) = 0) :
    - Space.distGrad (electricPotential q ε 0) = electricField q ε 0
```

lemma electricField_eq_neg_distGrad_electricPotential_origin [source]

```
lemma electricField_eq_neg_distGrad_electricPotential_origin (q ε : ℝ) :
    electricField q ε 0 = - Space.distGrad (electricPotential q ε 0)
```

lemma electricField_eq_neg_distGrad_electricPotential [source]

```
lemma electricField_eq_neg_distGrad_electricPotential (q ε : ℝ) (r₀ : Space 3) :
    electricField q ε r₀ = - Space.distGrad (electricPotential q ε r₀)
```

2.10 PhysLean.Electromagnetism.Vacuum.HarmonicWave

PhysLean/Electromagnetism/Vacuum/HarmonicWave.lean on GitHub.

def harmonicWaveX [source]

The electromagnetic potential for a Harmonic wave travelling in the `x`-direction with wave number `k`.

```
noncomputable def harmonicWaveX (𝓕 : FreeSpace) (k : ℝ) (E₀ : Fin d → ℝ)
  (φ : Fin d → ℝ) : ElectromagneticPotential d.succ
```

lemma harmonicWaveX_differentiable [source]

```
lemma harmonicWaveX_differentiable {d} (𝓕 : FreeSpace) (k : ℝ)
    (E₀ : Fin d → ℝ) (φ : Fin d → ℝ) :
    Differentiable ℝ (harmonicWaveX 𝓕 k E₀ φ)
```

lemma harmonicWaveX_contDiff [source]

```
lemma harmonicWaveX_contDiff {d} (n : WithTop ℕ∞) (𝓕 : FreeSpace) (k : ℝ)
    (E₀ : Fin d → ℝ) (φ : Fin d → ℝ) :
    ContDiff ℝ n (harmonicWaveX 𝓕 k E₀ φ)
```

lemma harmonicWaveX_scalarPotential_eq_zero [source]

```
@[simp]
lemma harmonicWaveX_scalarPotential_eq_zero {d} (𝓕 : FreeSpace) (k : ℝ)
    (E₀ : Fin d → ℝ) (φ : Fin d → ℝ) :
```

```
    (harmonicWaveX 𝓕 k E₀ φ).scalarPotential 𝓕.c = 0
```

lemma harmonicWaveX_vectorPotential_zero_eq_zero [source]

```
@[simp]
lemma harmonicWaveX_vectorPotential_zero_eq_zero {d} (𝓕 : FreeSpace) (k : ℝ)
    (E₀ : Fin d → ℝ) (φ : Fin d → ℝ) (t : Time) (x : Space d.succ) :
    (harmonicWaveX 𝓕 k E₀ φ).vectorPotential 𝓕.c t x 0 = 0
```

lemma harmonicWaveX_vectorPotential_succ [source]

```
lemma harmonicWaveX_vectorPotential_succ {d} (𝓕 : FreeSpace) (k : ℝ)
    (E₀ : Fin d → ℝ) (φ : Fin d → ℝ) (t : Time) (x : Space d.succ) (i : Fin d) :
    (harmonicWaveX 𝓕 k E₀ φ).vectorPotential 𝓕.c t x i.succ =
    - E₀ i * 1 / (𝓕.c * k) * Real.sin (k * (t.val * 𝓕.c - x 0) + φ i)
```

lemma harmonicWaveX_vectorPotential_succ' [source]

```
lemma harmonicWaveX_vectorPotential_succ' {d} (𝓕 : FreeSpace) (k : ℝ)
    (E₀ : Fin d → ℝ) (φ : Fin d → ℝ) (t : Time) (x : Space d.succ) (i : ℕ)
    (hi : i.succ < d.succ) :
    (harmonicWaveX 𝓕 k E₀ φ).vectorPotential 𝓕.c t x ⟨i.succ, hi⟩ =
    - E₀ ⟨i, by grind⟩ * 1 / (𝓕.c * k) * Real.sin (k * (t.val * 𝓕.c - x 0) + φ ⟨i, by grind⟩)
```

lemma harmonicWaveX_vectorPotential_space_deriv_succ [source]

```
@[simp]
lemma harmonicWaveX_vectorPotential_space_deriv_succ {d} (𝓕 : FreeSpace) (k : ℝ)
    (E₀ : Fin d → ℝ) (φ : Fin d → ℝ) (t : Time) (x : Space d.succ) (j : Fin d)
    (i : Fin d.succ) :
    Space.deriv j.succ (fun x => vectorPotential 𝓕.c (harmonicWaveX 𝓕 k E₀ φ) t x i) x
    = 0
```

lemma harmonicWaveX_vectorPotential_succ_space_deriv_zero [source]

```
@[simp]
lemma harmonicWaveX_vectorPotential_succ_space_deriv_zero {d} (𝓕 : FreeSpace) (k : ℝ) (hk : k
    ≠ 0)
    (E₀ : Fin d → ℝ) (φ : Fin d → ℝ) (t : Time) (x : Space d.succ) (i : Fin d) :
    Space.deriv 0 (fun x => vectorPotential 𝓕.c (harmonicWaveX 𝓕 k E₀ φ) t x i.succ) x
    = E₀ i / 𝓕.c.val * Real.cos (𝓕.c.val * k * t.val - k * x 0 + φ i)
```

lemma harmonicWaveX_electricField_zero [source]

```
lemma harmonicWaveX_electricField_zero {d} (𝓕 : FreeSpace) (k : ℝ)
    (E₀ : Fin d → ℝ) (φ : Fin d → ℝ) (t : Time) (x : Space d.succ) :
    (harmonicWaveX 𝓕 k E₀ φ).electricField 𝓕.c t x 0 = 0
```

lemma harmonicWaveX_electricField_succ [source]

```
lemma harmonicWaveX_electricField_succ {d} (𝓕 : FreeSpace) (k : ℝ) (hk : k ≠ 0)
    (E₀ : Fin d → ℝ) (φ : Fin d → ℝ) (t : Time) (x : Space d.succ) (i : Fin d) :
    (harmonicWaveX 𝓕 k E₀ φ).electricField 𝓕.c t x i.succ =
```

```
    E₀ i * Real.cos (k * 𝓕.c * t.val - k * x 0 + φ i)
```

lemma harmonicWaveX_electricField_space_deriv_same [source]

```
lemma harmonicWaveX_electricField_space_deriv_same {d} (𝓕 : FreeSpace) (k : ℝ) (hk : k ≠ 0)
    (E₀ : Fin d → ℝ) (φ : Fin d → ℝ) (t : Time) (x : Space d.succ) (i : Fin d.succ) :
    Space.deriv i (fun x => electricField 𝓕.c (harmonicWaveX 𝓕 k E₀ φ) t x i) x
    = 0
```

lemma harmonicWaveX_electricField_succ_time_deriv [source]

```
lemma harmonicWaveX_electricField_succ_time_deriv {d} (𝓕 : FreeSpace) (k : ℝ) (hk : k ≠ 0)
    (E₀ : Fin d → ℝ) (φ : Fin d → ℝ) (t : Time) (x : Space d.succ) (i : Fin d) :
    Time.deriv (fun t => electricField 𝓕.c (harmonicWaveX 𝓕 k E₀ φ) t x i.succ) t
    = - k * 𝓕.c * E₀ i * Real.sin (k * 𝓕.c * t.val - k * x 0 + φ i)
```

lemma harmonicWaveX_div_electricField_eq_zero [source]

```
@[simp]
lemma harmonicWaveX_div_electricField_eq_zero {d} (𝓕 : FreeSpace) (k : ℝ) (hk : k ≠ 0)
    (E₀ : Fin d → ℝ) (φ : Fin d → ℝ) (t : Time) (x : Space d.succ) :
    Space.div (fun x => electricField 𝓕.c (harmonicWaveX 𝓕 k E₀ φ) t x) x = 0
```

lemma harmonicWaveX_magneticFieldMatrix_succ_succ [source]

```
@[simp]
lemma harmonicWaveX_magneticFieldMatrix_succ_succ {d} (𝓕 : FreeSpace) (k : ℝ)
    (E₀ : Fin d → ℝ) (φ : Fin d → ℝ) (t : Time) (x : Space d.succ)
    (i j : Fin d) :
    (harmonicWaveX 𝓕 k E₀ φ).magneticFieldMatrix 𝓕.c t x (i.succ, j.succ) = 0
```

lemma harmonicWaveX_magneticFieldMatrix_zero_succ [source]

```
lemma harmonicWaveX_magneticFieldMatrix_zero_succ {d} (𝓕 : FreeSpace) (k : ℝ) (hk : k ≠ 0)
    (E₀ : Fin d → ℝ) (φ : Fin d → ℝ) (t : Time) (x : Space d.succ)
    (i : Fin d) :
    (harmonicWaveX 𝓕 k E₀ φ).magneticFieldMatrix 𝓕.c t x (0, i.succ) =
    (- E₀ i / 𝓕.c.val) * cos (𝓕.c.val * k * t.val - k * x 0 + φ i)
```

lemma harmonicWaveX_magneticFieldMatrix_succ_zero [source]

```
lemma harmonicWaveX_magneticFieldMatrix_succ_zero {d} (𝓕 : FreeSpace) (k : ℝ) (hk : k ≠ 0)
    (E₀ : Fin d → ℝ) (φ : Fin d → ℝ) (t : Time) (x : Space d.succ)
    (i : Fin d) :
    (harmonicWaveX 𝓕 k E₀ φ).magneticFieldMatrix 𝓕.c t x (i.succ, 0) =
    (E₀ i / 𝓕.c.val) * cos (𝓕.c.val * k * t.val - k * x 0 + φ i)
```

lemma harmonicWaveX_magneticFieldMatrix_space_deriv_succ [source]

```
lemma harmonicWaveX_magneticFieldMatrix_space_deriv_succ {d} (𝓕 : FreeSpace) (k : ℝ)
    (hk : k ≠ 0)
    (E₀ : Fin d → ℝ) (φ : Fin d → ℝ) (t : Time) (x : Space d.succ)
    (i j : Fin d.succ) (l : Fin d) :
```

```
    Space.deriv l.succ (fun x => magneticFieldMatrix 𝓕.c (harmonicWaveX 𝓕 k E₀ φ) t x (i, j))
    x
    = 0
```

lemma harmonicWaveX_magneticFieldMatrix_zero_succ_space_deriv_zero [source]

```
lemma harmonicWaveX_magneticFieldMatrix_zero_succ_space_deriv_zero {d} (𝓕 : FreeSpace) (k : ℝ)
    (hk : k ≠ 0)
    (E₀ : Fin d → ℝ) (φ : Fin d → ℝ) (t : Time) (x : Space d.succ)
    (i : Fin d) :
    Space.deriv 0 (fun x => magneticFieldMatrix 𝓕.c (harmonicWaveX 𝓕 k E₀ φ) t x (0, i.succ))
    x
    = -E₀ i * k / 𝓕.c.val * sin (𝓕.c.val * k * t.val - k * x 0 + φ i)
```

lemma harmonicWaveX_isExtrema [source]

```
lemma harmonicWaveX_isExtrema {d} (𝓕 : FreeSpace) (k : ℝ) (hk : k ≠ 0)
    (E₀ : Fin d → ℝ) (φ : Fin d → ℝ) :
    IsExtrema 𝓕 (harmonicWaveX 𝓕 k E₀ φ) 0
```

lemma harmonicWaveX_isPlaneWave [source]

```
lemma harmonicWaveX_isPlaneWave {d} (𝓕 : FreeSpace) (k : ℝ) (hk : k ≠ 0)
    (E₀ : Fin d → ℝ) (φ : Fin d → ℝ) :
    IsPlaneWave 𝓕 (harmonicWaveX 𝓕 k E₀ φ) (Space.basis 0, by simp)
```

lemma harmonicWaveX_polarization_ellipse [source]

```
lemma harmonicWaveX_polarization_ellipse {d} (𝓕 : FreeSpace) (k : ℝ) (hk : k ≠ 0)
    (E₀ : Fin d → ℝ) (φ : Fin d → ℝ) (t : Time) (x : Space d.succ) (hi : ∀ i, E₀ i ≠ 0) :
    2 * d * ∑ i : Fin d, ((harmonicWaveX 𝓕 k E₀ φ).electricField 𝓕.c t x i.succ / E₀ i) ^ 2 -
    2 * ∑ i, ∑ j, ((harmonicWaveX 𝓕 k E₀ φ).electricField 𝓕.c t x i.succ / E₀ i) *
      ((harmonicWaveX 𝓕 k E₀ φ).electricField 𝓕.c t x j.succ / E₀ j) *
      Real.cos (φ j - φ i) =
    ∑ i, ∑ j, Real.sin (φ j - φ i) ^ 2
```

2.11 PhysLean.Electromagnetism.Vacuum.IsPlaneWave

PhysLean/Electromagnetism/Vacuum/IsPlaneWave.lean on GitHub.

def IsPlaneWave [source]

The proposition on a electromagnetic potential which is true if it corresponds to a plane wave.

```
def IsPlaneWave {d : ℕ} (𝓕 : FreeSpace)
    (A : ElectromagneticPotential d) (s : Direction d) : Prop
```

def electricFunction [source]

The corresponding electric field function from ℝ to `EuclideanSpace ℝ (Fin d)` of a plane wave.

```
noncomputable def electricFunction {d : ℕ} {𝓕 : FreeSpace}
    {A : ElectromagneticPotential d} {s : Direction d}
    (hA : IsPlaneWave 𝓕 A s) : ℝ → EuclideanSpace ℝ (Fin d)
```

lemma electricField_eq_electricFunction [source]

```
lemma electricField_eq_electricFunction {d : ℕ} {𝓕 : FreeSpace}
    {A : ElectromagneticPotential d} {s : Direction d}
    (P : IsPlaneWave 𝓕 A s) (t : Time) (x : Space d) :
    A.electricField 𝓕.c t x =
    P.electricFunction (⟪x, s.unit⟫_ℝ - 𝓕.c * t)
```

def magneticFunction [source]

The corresponding magnetic field function from ℝ *to* `Fin d × Fin d → ℝ` *of a plane wave.*

```
noncomputable def magneticFunction {d : ℕ} {𝓕 : FreeSpace}
    {A : ElectromagneticPotential d} {s : Direction d}
    (hA : IsPlaneWave 𝓕 A s) : ℝ → Fin d × Fin d → ℝ
```

lemma magneticFieldMatrix_eq_magneticFunction [source]

```
lemma magneticFieldMatrix_eq_magneticFunction {d : ℕ}
    {𝓕 : FreeSpace} {A : ElectromagneticPotential d} {s : Direction d}
    (P : IsPlaneWave 𝓕 A s) (t : Time) (x : Space d) :
    A.magneticFieldMatrix 𝓕.c t x =
    P.magneticFunction (⟪x, s.unit⟫_ℝ - 𝓕.c * t)
```

lemma electricFunction_eq_electricField [source]

```
lemma electricFunction_eq_electricField {d : ℕ}
    {𝓕 : FreeSpace} {A : ElectromagneticPotential d}
    {s : Direction d} (P : IsPlaneWave 𝓕 A s) :
    P.electricFunction = fun u =>
    A.electricField 𝓕.c ⟨(- u)/𝓕.c.1⟩ (0 : Space d)
```

lemma magneticFunction_eq_magneticFieldMatrix [source]

```
lemma magneticFunction_eq_magneticFieldMatrix {d : ℕ}
    {𝓕 : FreeSpace} {A : ElectromagneticPotential d}
    {s : Direction d} (P : IsPlaneWave 𝓕 A s) :
    P.magneticFunction = fun u =>
    A.magneticFieldMatrix 𝓕.c ⟨(- u)/𝓕.c.1⟩ (0 : Space d)
```

lemma electricFunction_unique [source]

```
lemma electricFunction_unique {d : ℕ} {𝓕 : FreeSpace}
    {A : ElectromagneticPotential d} {s : Direction d}
    (P : IsPlaneWave 𝓕 A s) (E1 : ℝ → EuclideanSpace ℝ (Fin d))
    (hE₁ : A.electricField 𝓕.c = planeWave E1 𝓕.c s) :
    E1 = P.electricFunction
```

lemma magneticFunction_unique [source]

```
lemma magneticFunction_unique {d : ℕ} {𝓕 : FreeSpace}
    {A : ElectromagneticPotential d} {s : Direction d}
    (P : IsPlaneWave 𝓕 A s)
    (B1 : ℝ → Fin d × Fin d → ℝ)
    (hB₁ : ∀ t x, A.magneticFieldMatrix 𝓕.c t x =
      B1 (⟪x, s.unit⟫_ℝ - 𝓕.c * t)) :
    B1 = P.magneticFunction
```

lemma electricFunction_differentiable [source]

```
lemma electricFunction_differentiable {d : ℕ}
    {𝓕 : FreeSpace} {A : ElectromagneticPotential d}
    {s : Direction d} (P : IsPlaneWave 𝓕 A s) (hA : ContDiff ℝ 2 A) :
    Differentiable ℝ P.electricFunction
```

lemma magneticFunction_differentiable [source]

```
lemma magneticFunction_differentiable {d : ℕ}
    {𝓕 : FreeSpace} {A : ElectromagneticPotential d}
    {s : Direction d} (P : IsPlaneWave 𝓕 A s) (hA : ContDiff ℝ 2 A)
    (ij : Fin d × Fin d) :
    Differentiable ℝ (fun u => P.magneticFunction u ij)
```

lemma electricField_time_deriv [source]

```
lemma electricField_time_deriv {d : ℕ}
    {𝓕 : FreeSpace} {A : ElectromagneticPotential d}
    {s : Direction d} (P : IsPlaneWave 𝓕 A s) (hA : ContDiff ℝ 2 A) (t : Time)
    (x : Space d) :
    ∂ₜ (A.electricField 𝓕.c · x) t = - 𝓕.c.val •
    fderiv ℝ P.electricFunction (⟪x, s.unit⟫_ℝ - 𝓕.c.val * t.val) 1
```

lemma magneticFieldMatrix_time_deriv [source]

```
lemma magneticFieldMatrix_time_deriv {d : ℕ}
    {𝓕 : FreeSpace} {A : ElectromagneticPotential d}
    {s : Direction d} (P : IsPlaneWave 𝓕 A s) (hA : ContDiff ℝ 2 A) (t : Time)
    (x : Space d) (i j : Fin d) :
    ∂ₜ (A.magneticFieldMatrix 𝓕.c · x (i, j)) t = - 𝓕.c.val •
    fderiv ℝ (fun u => P.magneticFunction u (i, j)) (⟪x, s.unit⟫_ℝ - 𝓕.c.val * t.val) 1
```

lemma electricField_space_deriv [source]

```
lemma electricField_space_deriv {d : ℕ}
    {𝓕 : FreeSpace} {A : ElectromagneticPotential d}
    {s : Direction d} (P : IsPlaneWave 𝓕 A s) (hA : ContDiff ℝ 2 A) (t : Time)
    (x : Space d) (i : Fin d) :
    ∂[i] (A.electricField 𝓕.c t ·) x = s.unit i •
    fderiv ℝ P.electricFunction (⟪x, s.unit⟫_ℝ - 𝓕.c.val * t.val) 1
```

lemma magneticFieldMatrix_space_deriv [source]

```
lemma magneticFieldMatrix_space_deriv {d : ℕ}
    {𝓕 : FreeSpace} {A : ElectromagneticPotential d}
    {s : Direction d} (P : IsPlaneWave 𝓕 A s) (hA : ContDiff ℝ 2 A) (t : Time)
    (x : Space d) (i j : Fin d) (k : Fin d) :
    ∂[k] (A.magneticFieldMatrix 𝓕.c t · (i, j)) x = s.unit k •
    fderiv ℝ (fun u => P.magneticFunction u (i, j))
      (⟪x, s.unit⟫_ℝ - 𝓕.c.val * t.val) 1
```

lemma electricField_space_deriv_eq_time_deriv [source]

```
lemma electricField_space_deriv_eq_time_deriv {d : ℕ} {𝓕 : FreeSpace}
    {A : ElectromagneticPotential d}
    {s : Direction d} (P : IsPlaneWave 𝓕 A s) (hA : ContDiff ℝ 2 A) (t : Time)
    (x : Space d) (i : Fin d) (k : Fin d) :
    ∂[k] (A.electricField 𝓕.c t · i) x = - (s.unit k / 𝓕.c.val) •
    ∂ₜ (A.electricField 𝓕.c · x i) t
```

lemma magneticFieldMatrix_space_deriv_eq_time_deriv [source]

```
lemma magneticFieldMatrix_space_deriv_eq_time_deriv {d : ℕ}
    {𝓕 : FreeSpace} {A : ElectromagneticPotential d}
    {s : Direction d} (P : IsPlaneWave 𝓕 A s) (hA : ContDiff ℝ 2 A) (t : Time)
    (x : Space d) (i j : Fin d) (k : Fin d) :
    ∂[k] (A.magneticFieldMatrix 𝓕.c t · (i, j)) x = - (s.unit k / 𝓕.c.val) •
    ∂ₜ (A.magneticFieldMatrix 𝓕.c · x (i, j)) t
```

lemma time_deriv_magneticFieldMatrix_eq_electricField_mul_propogator [source]

```
lemma time_deriv_magneticFieldMatrix_eq_electricField_mul_propogator {d : ℕ}
    {𝓕 : FreeSpace} {A : ElectromagneticPotential d}
    {s : Direction d} (P : IsPlaneWave 𝓕 A s) (hA : ContDiff ℝ 2 A)
    (t : Time) (x : Space d) (i j : Fin d) :
    ∂ₜ (A.magneticFieldMatrix 𝓕.c · x (i, j)) t =
    ∂ₜ (fun t => s.unit j / 𝓕.c * A.electricField 𝓕.c t x i
    - s.unit i / 𝓕.c * A.electricField 𝓕.c t x j) t
```

lemma space_deriv_magneticFieldMatrix_eq_electricField_mul_propogator [source]

```
lemma space_deriv_magneticFieldMatrix_eq_electricField_mul_propogator {d : ℕ}
    {𝓕 : FreeSpace} {A : ElectromagneticPotential d}
    {s : Direction d} (P : IsPlaneWave 𝓕 A s) (hA : ContDiff ℝ 2 A)
    (t : Time) (x : Space d) (i j k : Fin d) :
    ∂[k] (A.magneticFieldMatrix 𝓕.c t · (i, j)) x =
    ∂[k] (fun x => s.unit j / 𝓕.c * A.electricField 𝓕.c t x i
    - s.unit i / 𝓕.c * A.electricField 𝓕.c t x j) x
```

lemma magneticFieldMatrix_eq_propogator_cross_electricField [source]

```
lemma magneticFieldMatrix_eq_propogator_cross_electricField {d : ℕ}
    {𝓕 : FreeSpace} {A : ElectromagneticPotential d}
    {s : Direction d} (P : IsPlaneWave 𝓕 A s) (hA : ContDiff ℝ 2 A) (i j : Fin d) :
    ∃ C, ∀ t x, A.magneticFieldMatrix 𝓕.c t x (i, j) =
    1/ 𝓕.c * (s.unit j * A.electricField 𝓕.c t x i -
      s.unit i * A.electricField 𝓕.c t x j) + C
```

lemma time_deriv_electricField_eq_magneticFieldMatrix [source]

```
lemma time_deriv_electricField_eq_magneticFieldMatrix {d : ℕ}
    {𝓕 : FreeSpace} {A : ElectromagneticPotential d}
    {s : Direction d} (P : IsPlaneWave 𝓕 A s) (hA : ContDiff ℝ ∞ A)
    (h : IsExtrema 𝓕 A 0)
    (t : Time) (x : Space d) (i : Fin d) :
    ∂ₜ (A.electricField 𝓕.c · x i) t =
    ∂ₜ (fun t => 𝓕.c * ∑ j, A.magneticFieldMatrix 𝓕.c t x (i, j) * s.unit j) t
```

lemma space_deriv_electricField_eq_magneticFieldMatrix [source]

```
lemma space_deriv_electricField_eq_magneticFieldMatrix {d : ℕ}
    {𝓕 : FreeSpace} {A : ElectromagneticPotential d}
    {s : Direction d} (P : IsPlaneWave 𝓕 A s) (hA : ContDiff ℝ ∞ A)
    (h : IsExtrema 𝓕 A 0)
    (t : Time) (x : Space d) (i k : Fin d) :
    ∂[k] (A.electricField 𝓕.c t · i) x =
    ∂[k] (fun x => 𝓕.c * ∑ j, A.magneticFieldMatrix 𝓕.c t x (i, j) * s.unit j) x
```

lemma electricField_eq_propogator_cross_magneticFieldMatrix [source]

```
lemma electricField_eq_propogator_cross_magneticFieldMatrix {d : ℕ}
    {𝓕 : FreeSpace} {A : ElectromagneticPotential d}
    {s : Direction d} (P : IsPlaneWave 𝓕 A s) (hA : ContDiff ℝ ∞ A)
    (h : IsExtrema 𝓕 A 0) (i : Fin d) :
    ∃ C, ∀ t x, A.electricField 𝓕.c t x i =
    𝓕.c * ∑ j, A.magneticFieldMatrix 𝓕.c t x (i, j) * s.unit j + C
```

2.12 PhysLean.Mathematics.InnerProductSpace.Basic

PhysLean/Mathematics/InnerProductSpace/Basic.lean on GitHub.

lemma ofLp_inner_left [source]

```
lemma ofLp_inner_left (x : E) (y : WithLp 2 E) : ⟪WithLp.ofLp y, x⟫ = ⟪y, WithLp.toLp 2 x⟫
```

lemma toLp_inner_left [source]

```
lemma toLp_inner_left (x : WithLp 2 E) (y : E) : ⟪WithLp.toLp 2 y, x⟫ = ⟪y, WithLp.ofLp x⟫
```

instance InnerProductSpace' 𝕜 (E × F) [source]

```
noncomputable
instance : InnerProductSpace' 𝕜 (E × F) where
```

2.13 PhysLean.Mathematics.VariationalCalculus.HasVarAdjDeriv

PhysLean/Mathematics/VariationalCalculus/HasVarAdjDeriv.lean on GitHub.

lemma grad [source]

```
protected lemma grad {d} (u : Space d → ℝ) (hu : ContDiff ℝ ∞ u) :
    HasVarAdjDerivAt
      (fun (φ : Space d → ℝ) x => Space.grad φ x)
      (fun ψ x => - Space.div ψ x) u
```

2.14 PhysLean.Mathematics.VariationalCalculus.HasVarAdjoint

PhysLean/Mathematics/VariationalCalculus/HasVarAdjoint.lean on GitHub.

lemma grad [source]

```
lemma grad {d} : HasVarAdjoint (fun (φ : Space d → ℝ) x => Space.grad φ x)
    (fun ψ x => - Space.div ψ x)
```

2.15 PhysLean.Mathematics.VariationalCalculus.IsLocalizedfunctionTransform

PhysLean/Mathematics/VariationalCalculus/IsLocalizedfunctionTransform.lean on GitHub.

lemma div_comp_repr [source]

```
lemma div_comp_repr {d} : IsLocalizedFunctionTransform fun (φ : Space d → Space d) x =>
    Space.div (Space.basis.repr ∘ φ) x
```

2.16 PhysLean.Particles.StandardModel.HiggsBoson.Basic

PhysLean/Particles/StandardModel/HiggsBoson/Basic.lean on GitHub.

lemma contDiff [source]

```
@[fun_prop]
lemma contDiff (φ : HiggsField) :
    ContDiff ℝ ⊤ φ
```

2.17 PhysLean.QFT.QED.AnomalyCancellation.Even.BasisLinear

PhysLean/QFT/QED/AnomalyCancellation/Even/BasisLinear.lean on GitHub.

lemma Pa'_elim_eq_iff [source]

```
lemma Pa'_elim_eq_iff (g g' : Fin n.succ → ℚ) (f f' : Fin n → ℚ) :
    Pa' (Sum.elim g f) = Pa' (Sum.elim g' f') ↔ Pa g f = Pa g' f'
```

2.18 PhysLean.QFT.QED.AnomalyCancellation.Odd.BasisLinear

PhysLean/QFT/QED/AnomalyCancellation/Odd/BasisLinear.lean on GitHub.

lemma Pa'_elim_eq_iff [source]

```
lemma Pa'_elim_eq_iff (g g' : Fin n.succ → ℚ) (f f' : Fin n.succ → ℚ) :
    Pa' (Sum.elim g f) = Pa' (Sum.elim g' f') ↔ Pa g f = Pa g' f'
```

2.19 PhysLean.QuantumMechanics.OneDimension.HarmonicOscillator.Examples

PhysLean/QuantumMechanics/OneDimension/HarmonicOscillator/Examples.lean on GitHub.

def Q [source]

A concrete harmonic oscillator with `m = 1`, `ω = 1`.

```
noncomputable def Q : HarmonicOscillator where
```

2.20 PhysLean.Relativity.Tensors.Basic

PhysLean/Relativity/Tensors/Basic.lean on GitHub.

instance FiniteDimensional k (S.Tensor c) [source]

```
instance {k : Type} [Field k] {C G : Type} [Group G] (S : TensorSpecies k C G)
    {c : Fin n → C} : FiniteDimensional k (S.Tensor c)
```

instance TopologicalSpace (S.Tensor c) [source]

```
instance {k : Type} [RCLike k] {C G : Type} [Group G] (S : TensorSpecies k C G)
    {c : Fin n → C} : TopologicalSpace (S.Tensor c)
```

instance IsTopologicalAddGroup (S.Tensor c) [source]

```
instance {k : Type} [RCLike k] {C G : Type} [Group G] (S : TensorSpecies k C G)
    {c : Fin n → C} : IsTopologicalAddGroup (S.Tensor c)
```

2.21 PhysLean.Relativity.Tensors.RealTensor.CoVector.Basic

PhysLean/Relativity/Tensors/RealTensor/CoVector/Basic.lean on GitHub.

def equivEuclid [source]

The equivalence between `CoVector d` and `EuclideanSpace ℝ (Fin 1 ⊕ Fin d)`.

```
def equivEuclid (d : ℕ) :
    CoVector d ≃ₗ[ℝ] EuclideanSpace ℝ (Fin 1 ⊕ Fin d)
```

instance Norm (CoVector d) [source]

```
instance (d : ℕ) : Norm (CoVector d) where
```

lemma norm_eq_equivEuclid [source]

```
lemma norm_eq_equivEuclid (d : ℕ) (v : CoVector d) :
    ‖v‖ = ‖equivEuclid d v‖
```

instance Inner ℝ (CoVector d) [source]

```
instance (d : ℕ) : Inner ℝ (CoVector d) where
```

lemma inner_eq_equivEuclid [source]

```
lemma inner_eq_equivEuclid (d : ℕ) (v w : CoVector d) :
    ⟪v, w⟫_ℝ = ⟪equivEuclid d v, equivEuclid d w⟫_ℝ
```

instance innerProductSpace [source]

The Euclidean inner product structure on `CoVector`.

```
instance innerProductSpace (d : ℕ) : InnerProductSpace ℝ (CoVector d) where
```

2.22 PhysLean.Relativity.Tensors.RealTensor.Vector.Basic

PhysLean/Relativity/Tensors/RealTensor/Vector/Basic.lean on GitHub.

def equivEuclid [source]

The equivalence between `Vector d` and `EuclideanSpace ℝ (Fin 1 ⊕ Fin d)`.

```
def equivEuclid (d : ℕ) :
    Vector d ≃ₗ[ℝ] EuclideanSpace ℝ (Fin 1 ⊕ Fin d)
```

lemma equivEuclid_apply [source]

```
@[simp]
lemma equivEuclid_apply (d : ℕ) (v : Vector d) (i : Fin 1 ⊕ Fin d) :
    equivEuclid d v i = v i
```

instance Norm (Vector d) [source]

```
instance (d : ℕ) : Norm (Vector d) where
```

lemma norm_eq_equivEuclid [source]

```
lemma norm_eq_equivEuclid (d : ℕ) (v : Vector d) :
    ‖v‖ = ‖equivEuclid d v‖
```

lemma abs_component_le_norm [source]

```
@[simp]
lemma abs_component_le_norm {d : ℕ} (v : Vector d) (i : Fin 1 ⊕ Fin d) :
    |v i| ≤ ‖v‖
```

instance Inner ℝ (Vector d) [source]

```
instance (d : ℕ) : Inner ℝ (Vector d) where
```

lemma inner_eq_equivEuclid [source]

```
lemma inner_eq_equivEuclid (d : ℕ) (v w : Vector d) :
    ⟪v, w⟫_ℝ = ⟪equivEuclid d v, equivEuclid d w⟫_ℝ
```

instance innerProductSpace [source]

The Euclidean inner product structure on `CoVector`.

```
instance innerProductSpace (d : ℕ) : InnerProductSpace ℝ (Vector d) where
```

lemma coord_continuous [source]

```
@[fun_prop]
lemma coord_continuous {d : ℕ} (i : Fin 1 ⊕ Fin d) :
    Continuous (fun v : Vector d => v i)
```

lemma coord_contDiff [source]

```
@[fun_prop]
lemma coord_contDiff {n} {d : ℕ} (i : Fin 1 ⊕ Fin d) :
    ContDiff ℝ n (fun v : Vector d => v i)
```

lemma coord_differentiable [source]

```
@[fun_prop]
lemma coord_differentiable {d : ℕ} (i : Fin 1 ⊕ Fin d) :
    Differentiable ℝ (fun v : Vector d => v i)
```

lemma coord_differentiableAt [source]

```
@[fun_prop]
lemma coord_differentiableAt {d : ℕ} (i : Fin 1 ⊕ Fin d) (v : Vector d) :
    DifferentiableAt ℝ (fun v : Vector d => v i) v
```

def euclidCLE [source]

The continous linear equivalence between `Vector d` and Euclidean space.

```
def euclidCLE (d : ℕ) : Vector d ≃L[ℝ] EuclideanSpace ℝ (Fin 1 ⊕ Fin d)
```

def equivPi [source]

The continous linear equivalence between `Vector d` and the corresponding `Pi` type.

```
def equivPi (d : ℕ) :
    Vector d ≃L[ℝ] Π (_ : Fin 1 ⊕ Fin d), ℝ
```

lemma equivPi_apply [source]

```
@[simp]
lemma equivPi_apply {d : ℕ} (v : Vector d) (i : Fin 1 ⊕ Fin d) :
    equivPi d v i = v i
```

lemma continuous_of_apply [source]

```
@[fun_prop]
lemma continuous_of_apply {d : ℕ} {α : Type*} [TopologicalSpace α]
```

```
    (f : α → Vector d)
    (h : ∀ i : Fin 1 ⊕ Fin d, Continuous (fun x => f x i)) :
    Continuous f
```

lemma differentiable_apply [source]

```
lemma differentiable_apply {d : ℕ} {α : Type*} [NormedAddCommGroup α] [NormedSpace ℝ α]
    (f : α → Vector d) :
    (∀ i : Fin 1 ⊕ Fin d, Differentiable ℝ (fun x => f x i)) ↔ Differentiable ℝ f
```

lemma contDiff_apply [source]

```
lemma contDiff_apply {n : WithTop ℕ∞} {d : ℕ} {α : Type*}
    [NormedAddCommGroup α] [NormedSpace ℝ α]
    (f : α → Vector d) :
    (∀ i : Fin 1 ⊕ Fin d, ContDiff ℝ n (fun x => f x i)) ↔ ContDiff ℝ n f
```

lemma fderiv_coord [source]

```
@[simp]
lemma fderiv_coord {d : ℕ} (μ : Fin 1 ⊕ Fin d) (x : Vector d) :
    fderiv ℝ (fun v : Vector d => v μ) x = coordCLM μ
```

lemma actionCLM_injective [source]

```
lemma actionCLM_injective {d : ℕ} (Λ : LorentzGroup d) :
    Function.Injective (actionCLM Λ)
```

lemma actionCLM_surjective [source]

```
lemma actionCLM_surjective {d : ℕ} (Λ : LorentzGroup d) :
    Function.Surjective (actionCLM Λ)
```

def spatialCLM [source]

The spatial part of a Lorentz vector as a continous linear map.

```
def spatialCLM (d : ℕ) : Vector d →L[ℝ] EuclideanSpace ℝ (Fin d) where
```

lemma spatialCLM_apply_eq_spatialPart [source]

```
lemma spatialCLM_apply_eq_spatialPart {d : ℕ} (v : Vector d) (i : Fin d) :
    spatialCLM d v i = spatialPart v i
```

lemma spatialCLM_basis_sum_inl [source]

```
@[simp]
lemma spatialCLM_basis_sum_inl {d : ℕ} :
    spatialCLM d (basis (Sum.inl 0)) = 0
```

lemma spatialCLM_basis_sum_inr [source]

```
@[simp]
lemma spatialCLM_basis_sum_inr {d : ℕ} (i : Fin d) :
    spatialCLM d (basis (Sum.inr i)) = EuclideanSpace.basisFun (Fin d) ℝ i
```

def temporalCLM [source]

The temporal part of a Lorentz vector as a continuous linear map.

```
def temporalCLM (d : ℕ) : Vector d →L[ℝ] ℝ
```

lemma temporalCLM_apply_eq_timeComponent [source]

```
lemma temporalCLM_apply_eq_timeComponent {d : ℕ} (v : Vector d) :
    temporalCLM d v = timeComponent v
```

lemma temporalCLM_basis_sum_inr [source]

```
@[simp]
lemma temporalCLM_basis_sum_inr {d : ℕ} (i : Fin d) :
    temporalCLM d (basis (Sum.inr i)) = 0
```

lemma temporalCLM_basis_sum_inl [source]

```
@[simp]
lemma temporalCLM_basis_sum_inl {d : ℕ} :
    temporalCLM d (basis (Sum.inl 0)) = 1
```

lemma basis_inner [source]

```
lemma basis_inner {d : ℕ} (μ : Fin 1 ⊕ Fin d) (p : Lorentz.Vector d) :
    ⟪Lorentz.Vector.basis μ, p⟫_ℝ = p μ
```

lemma inner_basis [source]

```
lemma inner_basis {d : ℕ} (p : Lorentz.Vector d) (μ : Fin 1 ⊕ Fin d) :
    ⟪p, Lorentz.Vector.basis μ⟫_ℝ = p μ
```

2.23 PhysLean.Relativity.Tensors.Tensorial

PhysLean/Relativity/Tensors/Tensorial.lean on GitHub.

instance SMul G M [source]

```
noncomputable instance (priority := high) smulAction [Tensorial S c M] : SMul G M where
```

instance DistribMulAction G M [source]

```
noncomputable instance (priority := high) distribMulAction [Tensorial S c M] :
    DistribMulAction G M where
```

instance SMulCommClass k G M [source]

```
instance [Tensorial S c M] : SMulCommClass k G M where
```

instance Tensorial S (Fin.append c c2) (M ⊗[k] M₂) [source]

```
noncomputable instance (priority := high) prod [Tensorial S c M] {n2 : ℕ} {c2 : Fin n2 → C}
    {M₂ : Type} [AddCommMonoid M₂] [Module k M₂] [Tensorial S c2 M₂] :
    Tensorial S (Fin.append c c2) (M ⊗[k] M₂) where
```

instance FiniteDimensional k M [source]

```
instance [Tensorial S c M] : FiniteDimensional k M
```

def toTensorCLM [source]

The map from a type carrying an Tensorial instance to tensors, as a continuous linear map.

```
def toTensorCLM [IsTopologicalAddGroup M]
    [ContinuousSMul k M] [Tensorial S c M] [T2Space M] : M ≃L[k] (S.Tensor c) where
```

def actionCLM [source]

The Lorentz action on types carrying a tensorial instance as a continuous linear map.

```
noncomputable def actionCLM (g : G) [IsTopologicalAddGroup M]
    [ContinuousSMul k M] [Tensorial S c M] [T2Space M] : M →L[k] M
```

lemma actionCLM_apply [source]

```
lemma actionCLM_apply {g : G} [IsTopologicalAddGroup M]
    [ContinuousSMul k M] [Tensorial S c M] [T2Space M] (m : M) :
    actionCLM S g m = g • m
```

2.24 PhysLean.SpaceAndTime.Space.Basic

PhysLean/SpaceAndTime/Space/Basic.lean on GitHub.

lemma eq_of_val [source]

```
lemma eq_of_val {d} {p q : Space d} (h : p.val = q.val) :
    p = q
```

lemma val_eq_iff [source]

```
@[simp]
lemma val_eq_iff {d} {p q : Space d} :
    p.val = q.val ↔ p = q
```

instance CoeFun (Space d) (fun _ => Fin d → ℝ) [source]

```
instance {d} : CoeFun (Space d) (fun _ => Fin d → ℝ) where
```

lemma eq_of_apply [source]

```
@[ext]
lemma eq_of_apply {d} {p q : Space d}
    (h : ∀ i : Fin d, p i = q i) : p = q
```

instance Add (Space d) [source]

```
instance {d} : Add (Space d) where
```

lemma add_val [source]

```
@[simp]
lemma add_val {d: ℕ} (x y : Space d) :
    (x + y).val = x.val + y.val
```

lemma add_apply [source]

```
@[simp]
lemma add_apply {d : ℕ} (x y : Space d) (i : Fin d) :
    (x + y) i = x i + y i
```

instance Zero (Space d) [source]

```
instance {d} : Zero (Space d) where
```

lemma zero_val [source]

```
@[simp]
lemma zero_val {d : ℕ} : (0 : Space d).val = fun _ => 0
```

lemma zero_apply [source]

```
@[simp]
lemma zero_apply {d : ℕ} (i : Fin d) :
    (0 : Space d) i = 0
```

instance AddCommMonoid (Space d) [source]

```
instance {d} : AddCommMonoid (Space d) where
```

lemma nsmul_val [source]

```
@[simp]
lemma nsmul_val {d : ℕ} (n : ℕ) (a : Space d) :
    (n • a).val = fun i => n • a.val i
```

lemma nsmul_apply [source]

```
@[simp]
lemma nsmul_apply {d : ℕ} (n : ℕ) (a : Space d) (i : Fin d) :
    (n • a) i = n • (a i)
```

instance SMul ℝ (Space d) [source]

```
instance {d} : SMul ℝ (Space d) where
```

lemma smul_val [source]

```
@[simp]
lemma smul_val {d : ℕ} (c : ℝ) (p : Space d) :
    (c • p).val = fun i => c * p.val i
```

lemma smul_apply [source]

```
@[simp]
lemma smul_apply {d : ℕ} (c : ℝ) (p : Space d) (i : Fin d) :
    (c • p) i = c * (p i)
```

instance Module ℝ (Space d) [source]

```
instance {d} : Module ℝ (Space d) where
```

instance VAdd (EuclideanSpace ℝ (Fin d)) (Space d) [source]

```
noncomputable instance : VAdd (EuclideanSpace ℝ (Fin d)) (Space d) where
```

lemma vadd_val [source]

```
@[simp]
lemma vadd_val {d} (v : EuclideanSpace ℝ (Fin d)) (s : Space d) :
    (v +ᵥ s).val = fun i => v i + s.val i
```

lemma vadd_apply [source]

```
@[simp]
lemma vadd_apply {d} (v : EuclideanSpace ℝ (Fin d))
    (s : Space d) (i : Fin d) :
    (v +ᵥ s) i = v i + s i
```

lemma vadd_transitive [source]

```
lemma vadd_transitive {d} (s1 s2 : Space d) :
    ∃ v : EuclideanSpace ℝ (Fin d), v +ᵥ s1 = s2
```

lemma eq_vadd_zero [source]

```
lemma eq_vadd_zero {d} (s : Space d) :
    ∃ v : EuclideanSpace ℝ (Fin d), s = v +ᵥ (0 : Space d)
```

lemma smul_vadd_zero [source]

```
@[simp]
lemma smul_vadd_zero {d} (k : ℝ) (v : EuclideanSpace ℝ (Fin d)) :
    k • (v +ᵥ (0 : Space d)) = (k • v) +ᵥ (0 : Space d)
```

instance AddAction (EuclideanSpace ℝ (Fin d)) (Space d) [source]

```
noncomputable instance : AddAction (EuclideanSpace ℝ (Fin d)) (Space d) where
```

lemma add_vadd_zero [source]

```
@[simp]
lemma add_vadd_zero {d} (v1 v2 : EuclideanSpace ℝ (Fin d)) :
    (v1 +ᵥ (0 : Space d)) + (v2 +ᵥ (0 : Space d)) = (v1 + v2) +ᵥ (0 : Space d)
```

instance Norm (Space d) [source]

```
noncomputable instance {d} : Norm (Space d) where
```

lemma norm_eq [source]

```
lemma norm_eq {d} (p : Space d) : ‖p‖ = √ (∑ i, (p i) ^ 2)
```

lemma abs_eval_le_norm [source]

```
@[simp]
lemma abs_eval_le_norm {d} (p : Space d) (i : Fin d) :
    |p i| ≤ ‖p‖
```

lemma norm_sq_eq [source]

```
lemma norm_sq_eq {d} (p : Space d) :
    ‖p‖ ^ 2 = ∑ i, (p i) ^ 2
```

lemma norm_vadd_zero [source]

```
@[simp]
lemma norm_vadd_zero {d} (v : EuclideanSpace ℝ (Fin d)) :
    ‖v +ᵥ (0 : Space d)‖ = ‖v‖
```

instance Neg (Space d) [source]

```
instance : Neg (Space d) where
```

lemma neg_val [source]

```
@[simp]
lemma neg_val {d : ℕ} (p : Space d) :
    (-p).val = fun i => - (p.val i)
```

lemma neg_apply [source]

```
@[simp]
lemma neg_apply {d : ℕ} (p : Space d) (i : Fin d) :
    (-p) i = - (p i)
```

instance AddCommGroup (Space d) [source]

```
noncomputable instance {d} : AddCommGroup (Space d) where
```

lemma sub_apply [source]

```
@[simp]
lemma sub_apply {d} (p q : Space d) (i : Fin d) :
    (p - q) i = p i - q i
```

lemma sub_val [source]

```
@[simp]
lemma sub_val {d} (p q : Space d) :
    (p - q).val = fun i => p.val i - q.val i
```

lemma vadd_zero_sub_vadd_zero [source]

```
@[simp]
lemma vadd_zero_sub_vadd_zero {d} (v1 v2 : EuclideanSpace ℝ (Fin d)) :
    (v1 +ᵥ (0 : Space d)) - (v2 +ᵥ (0 : Space d)) = (v1 - v2) +ᵥ (0 : Space d)
```

instance SeminormedAddCommGroup (Space d) [source]

```
noncomputable instance {d} : SeminormedAddCommGroup (Space d) where
```

lemma dist_eq [source]

```
@[simp]
lemma dist_eq {d} (p q : Space d) :
    dist p q = ‖p - q‖
```

instance NormedAddCommGroup (Space d) [source]

```
noncomputable instance : NormedAddCommGroup (Space d) where
```

instance Inner ℝ (Space d) [source]

```
instance {d} : Inner ℝ (Space d) where
```

lemma inner_vadd_zero [source]

```
@[simp]
lemma inner_vadd_zero {d} (v1 v2 : EuclideanSpace ℝ (Fin d)) :
    inner ℝ (v1 +ᵥ (0 : Space d)) (v2 +ᵥ (0 : Space d)) = Inner.inner ℝ v1 v2
```

lemma inner_apply [source]

```
lemma inner_apply {d} (p q : Space d) :
    inner ℝ p q = ∑ i, p i * q i
```

instance InnerProductSpace ℝ (Space d) [source]

```
instance {d} : InnerProductSpace ℝ (Space d) where
```

instance MeasurableSpace (Space d) [source]

```
instance {d : ℕ} : MeasurableSpace (Space d)
```

instance BorelSpace (Space d) [source]

```
instance {d : ℕ} : BorelSpace (Space d) where
```

lemma apply_eq_basis_repr_apply [source]

```
lemma apply_eq_basis_repr_apply {d} (p : Space d) (i : Fin d) :
    p i = basis.repr p i
```

lemma basis_repr_apply [source]

```
@[simp]
lemma basis_repr_apply {d} (p : Space d) (i : Fin d) :
    basis.repr p i = p i
```

lemma basis_repr_symm_apply [source]

```
@[simp]
lemma basis_repr_symm_apply {d} (v : EuclideanSpace ℝ (Fin d)) (i : Fin d) :
    basis.repr.symm v i = v i
```

lemma basis_repr_inner_eq [source]

```
lemma basis_repr_inner_eq {d} (p : Space d) (v : EuclideanSpace ℝ (Fin d)) :
    ⟪basis.repr p, v⟫_ℝ = ⟪p, basis.repr.symm v⟫_ℝ
```

instance FiniteDimensional ℝ (Space d) [source]

```
instance {d : ℕ} : FiniteDimensional ℝ (Space d)
```

lemma finrank_eq_dim [source]

```
@[simp]
lemma finrank_eq_dim {d : ℕ} : Module.finrank ℝ (Space d) = d
```

lemma rank_eq_dim [source]

```
@[simp]
lemma rank_eq_dim {d : ℕ} : Module.rank ℝ (Space d) = d
```

lemma fderiv_basis_repr [source]

```
@[simp]
lemma fderiv_basis_repr {d} (p : Space d) :
    fderiv ℝ basis.repr p = basis.repr.toContinuousLinearMap
```

lemma fderiv_basis_repr_symm [source]

```
@[simp]
lemma fderiv_basis_repr_symm {d} (v : EuclideanSpace ℝ (Fin d)) :
    fderiv ℝ basis.repr.symm v = basis.repr.symm.toContinuousLinearMap
```

lemma eval_continuous [source]

```
@[fun_prop]
lemma eval_continuous {d} (i : Fin d) :
    Continuous (fun p : Space d => p i)
```

lemma eval_differentiable [source]

```
@[fun_prop]
lemma eval_differentiable {d} (i : Fin d) :
    Differentiable ℝ (fun p : Space d => p i)
```

lemma eval_contDiff [source]

```
@[fun_prop]
lemma eval_contDiff {d n} (i : Fin d) :
    ContDiff ℝ n (fun p : Space d => p i)
```

def equivPi [source]

The continous linear equivalence between `Space d` and the corresponding `Pi` type.

```
def equivPi (d : ℕ) :
    Space d ≃L[ℝ] Π (_ : Fin d), ℝ
```

lemma mk_continuous [source]

```
@[fun_prop]
lemma mk_continuous {d : ℕ} :
    Continuous (fun (f : Fin d → ℝ) => (⟨f⟩ : Space d))
```

lemma mk_differentiable [source]

```
@[fun_prop]
lemma mk_differentiable {d : ℕ} :
    Differentiable ℝ (fun (f : Fin d → ℝ) => (⟨f⟩ : Space d))
```

lemma mk_contDiff [source]

```
@[fun_prop]
lemma mk_contDiff {d n : ℕ} :
    ContDiff ℝ n (fun (f : Fin d → ℝ) => ((f) : Space d))
```

lemma fderiv_mk [source]

```
@[simp]
lemma fderiv_mk {d : ℕ} (f : Fin d → ℝ) :
    fderiv ℝ Space.mk f = (equivPi d).symm
```

lemma fderiv_val [source]

```
@[simp]
lemma fderiv_val {d : ℕ} (p : Space d) :
    fderiv ℝ Space.val p = (equivPi d)
```

lemma volume_eq_addHaar [source]

```
lemma volume_eq_addHaar {d} : (volume (α := Space d)) = Space.basis.toBasis.addHaar
```

lemma volume_metricBall_three [source]

```
@[simp]
lemma volume_metricBall_three :
    volume (Metric.ball (0 : Space 3) 1) = ENNReal.ofReal (4 / 3 * Real.pi)
```

instance Nontrivial (Space d.succ) [source]

```
instance {d : ℕ} : Nontrivial (Space d.succ)
```

instance Subsingleton (Space 0) [source]

```
instance : Subsingleton (Space 0)
```

lemma volume_closedBall_neq_zero [source]

```
lemma volume_closedBall_neq_zero {d : ℕ} (x : Space d.succ) (r : ℝ) (hr : 0 < r) :
    volume (Metric.closedBall x r) ≠ 0
```

lemma volume_closedBall_neq_top [source]

```
lemma volume_closedBall_neq_top {d : ℕ} (x : Space d.succ) (r : ℝ) :
    volume (Metric.closedBall x r) ≠ ⊤
```

2.25 PhysLean.SpaceAndTime.Space.ConstantSliceDist

PhysLean/SpaceAndTime/Space/ConstantSliceDist.lean on GitHub.

lemma schwartzMap_slice_bound [source]

```
lemma schwartzMap_slice_bound {n m} {d : ℕ} (i : Fin d.succ) :
    ∃ rt, ∀ (η : 𝓢(Space d.succ, ℝ)), ∃ k,
    Integrable (fun x : ℝ => ‖((1 + ‖x‖) ^ rt)⁻¹‖) volume ∧
    (∀ (x : Space d), ∀ r, ‖(slice i).symm (r, x)‖ ^ m *
      ‖iteratedFDeriv ℝ n ⇑η ((slice i).symm (r, x))‖ ≤ k * ‖(1 + ‖r‖) ^ (rt)‖⁻¹)
      ∧ k = (2 ^ (rt + m, n).1 * ((Finset.Iic (rt + m, n)).sup
        fun m => SchwartzMap.seminorm ℝ m.1 m.2) η)
```

lemma schwartzMap_mul_iteratedFDeriv_integrable_slice_symm [source]

```
@[fun_prop]
lemma schwartzMap_mul_iteratedFDeriv_integrable_slice_symm {d : ℕ} (n m : ℕ)
    (η : 𝓢(Space d.succ, ℝ))
    (x : Space d) (i : Fin d.succ) :
    Integrable (fun r => ‖(slice i).symm (r, x)‖ ^ m *
    ‖iteratedFDeriv ℝ n ⇑η ((slice i).symm (r, x))‖) volume
```

lemma schwartzMap_integrable_slice_symm [source]

```
lemma schwartzMap_integrable_slice_symm {d : ℕ} (i : Fin d.succ) (η : 𝓢(Space d.succ, ℝ))
    (x : Space d) : Integrable (fun r => η ((slice i).symm (r, x))) volume
```

lemma schwartzMap_fderiv_integrable_slice_symm [source]

```
lemma schwartzMap_fderiv_integrable_slice_symm {d : ℕ} (η : 𝓢(Space d.succ, ℝ)) (x : Space d)
    (i : Fin d.succ) :
    Integrable (fun r => fderiv ℝ (fun x => η (((slice i).symm (r, x)))) x) volume
```

lemma schwartzMap_fderiv_left_integrable_slice_symm [source]

```
@[fun_prop]
lemma schwartzMap_fderiv_left_integrable_slice_symm {d : ℕ} (η : 𝓢(Space d.succ, ℝ)) (x : Space
     d)
    (i : Fin d.succ) :
    Integrable (fun r => fderiv ℝ (fun r => η (((slice i).symm (r, x)))) r 1) volume
```

lemma schwartzMap_iteratedFDeriv_norm_slice_symm_integrable [source]

```
@[fun_prop]
lemma schwartzMap_iteratedFDeriv_norm_slice_symm_integrable {n} {d : ℕ} (η : 𝓢(Space d.succ, ℝ)
    )
    (x : Space d) (i : Fin d.succ) :
    Integrable (fun r => ‖iteratedFDeriv ℝ n ⇑η (((slice i).symm (r, x)))‖) volume
```

lemma schwartzMap_iteratedFDeriv_slice_symm_integrable [source]

```
@[fun_prop]
lemma schwartzMap_iteratedFDeriv_slice_symm_integrable {n} {d : ℕ} (η : 𝓢(Space d.succ, ℝ))
    (x : Space d) (i : Fin d.succ) :
    Integrable (fun r => iteratedFDeriv ℝ n ⇑η (((slice i).symm (r, x)))) volume
```

lemma continuous_schwartzMap_slice_integral [source]

```
lemma continuous_schwartzMap_slice_integral {d} (i : Fin d.succ) (η : 𝓢(Space d.succ, ℝ)) :
    Continuous (fun x : Space d => ∫ r : ℝ, η ((slice i).symm (r, x)))
```

lemma schwartzMap_slice_integral_hasFDerivAt [source]

```
lemma schwartzMap_slice_integral_hasFDerivAt {d : ℕ} (η : 𝓢(Space d.succ, ℝ)) (i : Fin d.succ)
    (x₀ : Space d) :
    HasFDerivAt (fun x => ∫ (r : ℝ), η ((slice i).symm (r, x)))
      (∫ (r : ℝ), fderiv ℝ (fun x : Space d => η ((slice i).symm (r, x))) x₀) x₀
```

lemma schwartzMap_slice_integral_differentiable [source]

```
lemma schwartzMap_slice_integral_differentiable {d : ℕ} (η : 𝓢(Space d.succ, ℝ))
    (i : Fin d.succ) :
    Differentiable ℝ (fun x => ∫ (r : ℝ), η ((slice i).symm (r, x)))
```

lemma schwartzMap_slice_integral_contDiff [source]

```
lemma schwartzMap_slice_integral_contDiff {d : ℕ} (n : ℕ) (η : 𝓢(Space d.succ, ℝ))
    (i : Fin d.succ) :
    ContDiff ℝ n (fun x => ∫ (r : ℝ), η ((slice i).symm (r, x)))
```

lemma schwartzMap_slice_integral_iteratedFDeriv_apply [source]

```
lemma schwartzMap_slice_integral_iteratedFDeriv_apply {d : ℕ} (n : ℕ) (η : 𝓢(Space d.succ, ℝ))
    (i : Fin d.succ) :
    ∀ x, ∀ y, iteratedFDeriv ℝ n (fun x => ∫ (r : ℝ), η ((slice i).symm (r, x))) x y =
      ∫ (r : ℝ), (iteratedFDeriv ℝ n η ((slice i).symm (r, x)))
      (fun j => (slice i).symm (0, y j))
```

lemma schwartzMap_slice_integral_iteratedFDeriv [source]

```
lemma schwartzMap_slice_integral_iteratedFDeriv {d : ℕ} (n : ℕ) (η : 𝓢(Space d.succ, ℝ))
    (i : Fin d.succ) (x : Space d) :
    iteratedFDeriv ℝ n (fun x => ∫ (r : ℝ), η ((slice i).symm (r, x))) x
    = (∫ (r : ℝ), iteratedFDeriv ℝ n η ((slice i).symm (r, x))).compContinuousLinearMap
      (fun _ => (slice i).symm.toContinuousLinearMap.comp
      (ContinuousLinearMap.prod (0 : Space d →L[ℝ] ℝ) (ContinuousLinearMap.id ℝ (Space d))))
```

lemma schwartzMap_slice_integral_iteratedFDeriv_norm_le [source]

```
lemma schwartzMap_slice_integral_iteratedFDeriv_norm_le {d : ℕ} (n : ℕ) (η : 𝓢(Space d.succ, ℝ)
    )
    (i : Fin d.succ) (x : Space d) :
    ‖iteratedFDeriv ℝ n (fun x => ∫ (r : ℝ), η ((slice i).symm (r, x))) x‖ ≤
        (∫ (r : ℝ), ‖iteratedFDeriv ℝ n η ((slice i).symm (r, x))‖) *
        ‖(slice i).symm.toContinuousLinearMap.comp
          (ContinuousLinearMap.prod (0 : Space d →L[ℝ] ℝ)
          (ContinuousLinearMap.id ℝ (Space d)))‖ ^ n
```

lemma schwartzMap_mul_pow_slice_integral_iteratedFDeriv_norm_le [source]

```
lemma schwartzMap_mul_pow_slice_integral_iteratedFDeriv_norm_le {d : ℕ} (n m : ℕ) (i : Fin d.
    succ) :
    ∃ rt, ∀ (η : 𝓢(Space d.succ, ℝ)),∀ (x : Space d),
    Integrable (fun x : ℝ => ‖((1 + ‖x‖) ^ rt)⁻¹‖) volume ∧
    ‖x‖ ^ m * ‖iteratedFDeriv ℝ n (fun x => ∫ (r : ℝ), η ((slice i).symm (r, x))) x‖ ≤
        ((∫ (r : ℝ), ‖((1 + ‖r‖) ^ rt)⁻¹‖) *
        ‖(slice i).symm.toContinuousLinearMap.comp
          (ContinuousLinearMap.prod (0 : Space d →L[ℝ] ℝ)
          (ContinuousLinearMap.id ℝ (Space d)))‖ ^ n)
        * (2 ^ (rt + m, n).1 * ((Finset.Iic (rt + m, n)).sup
          fun m => SchwartzMap.seminorm ℝ m.1 m.2) η)
```

def sliceSchwartz [source]

The continuous linear map taking a Schwartz map and integrating over the `i`th component, to give a Schwartz map of one dimension lower.

```
def sliceSchwartz {d : ℕ} (i : Fin d.succ) :
    𝓢(Space d.succ, ℝ) →L[ℝ] 𝓢(Space d, ℝ)
```

lemma sliceSchwartz_apply [source]

```
lemma sliceSchwartz_apply {d : ℕ} (i : Fin d.succ) (η : 𝓢(Space d.succ, ℝ)) (x : Space d) :
    sliceSchwartz i η x = ∫ (r : ℝ), η ((slice i).symm (r, x))
```

def constantSliceDist [source]

Distributions on `Space d.succ` from distributions on `Space d` given a direction `i`. These distributions are constant on slices in the `i` direction..

```
def constantSliceDist {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M] {d : ℕ} (i : Fin d.
    succ) :
    ((Space d) →d[ℝ] M) →ₗ[ℝ] (Space d.succ) →d[ℝ] M where
```

lemma constantSliceDist_apply [source]

```
lemma constantSliceDist_apply {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M]
    {d : ℕ} (i : Fin d.succ) (f : (Space d) →d[ℝ] M) (η : 𝓢(Space d.succ, ℝ)) :
    constantSliceDist i f η = f (sliceSchwartz i η)
```

lemma distDeriv_constantSliceDist_same [source]

```
lemma distDeriv_constantSliceDist_same {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M]
    {d : ℕ} (i : Fin d.succ) (f : (Space d) →d[ℝ] M) :
    distDeriv i (constantSliceDist i f) = 0
```

lemma distDeriv_constantSliceDist_succAbove [source]

```
lemma distDeriv_constantSliceDist_succAbove {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M]
    {d : ℕ} (i : Fin d.succ) (j : Fin d) (f : (Space d) →d[ℝ] M) :
    distDeriv (i.succAbove j) (constantSliceDist i f) =
    constantSliceDist i (distDeriv j f)
```

2.26 PhysLean.SpaceAndTime.Space.CrossProduct

PhysLean/SpaceAndTime/Space/CrossProduct.lean on GitHub.

lemma fderiv_cross_commute [source]

Cross product and fderiv commute.

```
lemma fderiv_cross_commute {t : Time} {s : EuclideanSpace ℝ (Fin 3)}
    {f : Time → EuclideanSpace ℝ (Fin 3)} (hf : Differentiable ℝ f) :
    s ×ₑ₃ (fderiv ℝ (fun t' => f t') t) 1
    = fderiv ℝ (fun t' => s ×ₑ₃ (f t')) t 1
```

lemma time_deriv_cross_commute [source]

Cross product and time derivative commute.

```
lemma time_deriv_cross_commute {s : EuclideanSpace ℝ (Fin 3)} {f : Time → EuclideanSpace ℝ (Fin
     3)}
    (hf : Differentiable ℝ f) :
    s ×ₑ₃ (∂ₜ (fun t => f t) t) = ∂ₜ (fun t => s ×ₑ₃ (f t)) t
```

lemma inner_cross_self [source]

```
lemma inner_cross_self (v w : EuclideanSpace ℝ (Fin 3)) :
    inner ℝ v (w ×ₑ₃ v) = 0
```

lemma inner_self_cross [source]

```
lemma inner_self_cross (v w : EuclideanSpace ℝ (Fin 3)) :
    inner ℝ v (v ×ₑ₃ w) = 0
```

2.27 PhysLean.SpaceAndTime.Space.Derivatives.Basic

PhysLean/SpaceAndTime/Space/Derivatives/Basic.lean on GitHub.

lemma fderiv_eq_sum_deriv [source]

```
lemma fderiv_eq_sum_deriv {M d} [AddCommGroup M] [Module ℝ M] [TopologicalSpace M]
    (f : Space d → M) (x y : Space d) :
    fderiv ℝ f x y = ∑ i : Fin d, y i • ∂[i] f x
```

lemma inner_differentiableAt [source]

```
@[fun_prop]
lemma inner_differentiableAt {d : ℕ} (x : Space d) :
    DifferentiableAt ℝ (fun y : Space d => ⟪y, y⟫_ℝ) x
```

lemma inner_apply_differentiableAt [source]

```
@[fun_prop]
lemma inner_apply_differentiableAt {d : ℕ} [NormedAddCommGroup M]
    [NormedSpace ℝ M]
    {f : M → Space d} {g : M → Space d} (x : M)
```

```
    (hf : DifferentiableAt ℝ f x) (hg : DifferentiableAt ℝ g x) :
    DifferentiableAt ℝ (fun y : M => ⟪f y, g y⟫_ℝ) x
```

lemma inner_apply_differentiable [source]

```
@[fun_prop]
lemma inner_apply_differentiable {d : ℕ} [NormedAddCommGroup M]
    [NormedSpace ℝ M]
    {f : M → Space d} {g : M → Space d}
    (hf : Differentiable ℝ f) (hg : Differentiable ℝ g) :
    Differentiable ℝ (fun y : M => ⟪f y, g y⟫_ℝ)
```

lemma inner_contDiff [source]

```
@[fun_prop]
lemma inner_contDiff {n : WithTop ℕ∞} {d : ℕ} :
    ContDiff ℝ n (fun y : Space d => ⟪y, y⟫_ℝ)
```

lemma inner_apply_contDiff [source]

```
@[fun_prop]
lemma inner_apply_contDiff {n : WithTop ℕ∞} {d : ℕ} [NormedAddCommGroup M]
    [NormedSpace ℝ M]
    {f : M → Space d} {g : M → Space d}
    (hf : ContDiff ℝ n f) (hg : ContDiff ℝ n g) :
    ContDiff ℝ n (fun y : M => ⟪f y, g y⟫_ℝ)
```

lemma deriv_inner_left [source]

```
@[simp]
lemma deriv_inner_left {d} (x1 x2 : Space d) (i : Fin d) :
    deriv i (fun x => ⟪x, x2⟫_ℝ) x1 = x2 i
```

lemma deriv_inner_right [source]

```
@[simp]
lemma deriv_inner_right {d} (x1 x2 : Space d) (i : Fin d) :
    deriv i (fun x => ⟪x1, x⟫_ℝ) x2 = x1 i
```

def distDeriv [source]

Given a distribution (function) `f : Space d →d[ℝ] M` the derivative of `f` in direction `μ`.

```
noncomputable def distDeriv {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (μ : Fin d) : ((Space d) →d[ℝ] M) →ₗ[ℝ] (Space d) →d[ℝ] M where
```

lemma distDeriv_apply [source]

```
lemma distDeriv_apply {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (μ : Fin d) (f : (Space d) →d[ℝ] M) (ε : 𝓢(Space d, ℝ)) :
    (distDeriv μ f) ε = fderivD ℝ f ε (basis μ)
```

lemma distDeriv_commute [source]

```
lemma distDeriv_commute {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (μ ν : Fin d) (f : (Space d) →d[ℝ] M) :
    (distDeriv ν (distDeriv μ f)) = (distDeriv μ (distDeriv ν f))
```

2.28 PhysLean.SpaceAndTime.Space.Derivatives.Curl

PhysLean/SpaceAndTime/Space/Derivatives/Curl.lean on GitHub.

def distCurl [source]

The curl of a distribution `Space →d[ℝ] (EuclideanSpace ℝ (Fin 3))` as a distribution `Space →d[ℝ] (EuclideanSpace ℝ (Fin 3))`.

```
noncomputable def distCurl : (Space →d[ℝ] (EuclideanSpace ℝ (Fin 3))) →ₗ[ℝ]
    (Space) →d[ℝ] (EuclideanSpace ℝ (Fin 3)) where
```

lemma distCurl_apply_zero [source]

```
lemma distCurl_apply_zero (f : Space →d[ℝ] (EuclideanSpace ℝ (Fin 3))) (η : 𝓢(Space, ℝ)) :
    distCurl f η 0 = - f (SchwartzMap.evalCLM (𝕜 := ℝ) (basis 2) (fderivCLM ℝ η)) 1
    + f (SchwartzMap.evalCLM (𝕜 := ℝ) (basis 1) (fderivCLM ℝ η)) 2
```

lemma distCurl_apply_one [source]

```
lemma distCurl_apply_one (f : Space →d[ℝ] (EuclideanSpace ℝ (Fin 3))) (η : 𝓢(Space, ℝ)) :
    distCurl f η 1 = - f (SchwartzMap.evalCLM (𝕜 := ℝ) (basis 0) (fderivCLM ℝ η)) 2
    + f (SchwartzMap.evalCLM (𝕜 := ℝ) (basis 2) (fderivCLM ℝ η)) 0
```

lemma distCurl_apply_two [source]

```
lemma distCurl_apply_two (f : Space →d[ℝ] (EuclideanSpace ℝ (Fin 3))) (η : 𝓢(Space, ℝ)) :
    distCurl f η 2 = - f (SchwartzMap.evalCLM (𝕜 := ℝ) (basis 1) (fderivCLM ℝ η)) 0
    + f (SchwartzMap.evalCLM (𝕜 := ℝ) (basis 0) (fderivCLM ℝ η)) 1
```

lemma distCurl_apply [source]

```
lemma distCurl_apply (f : Space →d[ℝ] (EuclideanSpace ℝ (Fin 3))) (η : 𝓢(Space, ℝ)) :
    distCurl f η = WithLp.toLp 2 fun
    | 0 => - f (SchwartzMap.evalCLM (𝕜 := ℝ) (basis 2) (fderivCLM ℝ η)) 1
      + f (SchwartzMap.evalCLM (𝕜 := ℝ) (basis 1) (fderivCLM ℝ η)) 2
    | 1 => - f (SchwartzMap.evalCLM (𝕜 := ℝ) (basis 0) (fderivCLM ℝ η)) 2
      + f (SchwartzMap.evalCLM (𝕜 := ℝ) (basis 2) (fderivCLM ℝ η)) 0
    | 2 => - f (SchwartzMap.evalCLM (𝕜 := ℝ) (basis 1) (fderivCLM ℝ η)) 0
      + f (SchwartzMap.evalCLM (𝕜 := ℝ) (basis 0) (fderivCLM ℝ η)) 1
```

lemma distCurl_distGrad_eq_zero [source]

The curl of a grad is equal to zero.

```
@[simp]
lemma distCurl_distGrad_eq_zero (f : (Space) →d[ℝ] ℝ) :
    distCurl (distGrad f) = 0
```

2.29 PhysLean.SpaceAndTime.Space.Derivatives.Div

PhysLean/SpaceAndTime/Space/Derivatives/Div.lean on GitHub.

def distDiv [source]

The divergence of a distribution `(Space d) →d[ℝ] (EuclideanSpace ℝ (Fin d))` as a distribution `(Space d) →d[ℝ] ℝ`.

```
noncomputable def distDiv {d} :
    ((Space d) →d[ℝ] (EuclideanSpace ℝ (Fin d))) →ₗ[ℝ] (Space d) →d[ℝ] ℝ where
```

lemma distDiv_apply_eq_sum_fderivD [source]

```
lemma distDiv_apply_eq_sum_fderivD {d}
    (f : (Space d) →d[ℝ] EuclideanSpace ℝ (Fin d)) (η : 𝓢(Space d, ℝ)) :
    distDiv f η = ∑ i, fderivD ℝ f η (basis i) i
```

lemma distDiv_apply_eq_sum_distDeriv [source]

```
lemma distDiv_apply_eq_sum_distDeriv {d}
    (f : (Space d) →d[ℝ] EuclideanSpace ℝ (Fin d)) (η : 𝓢(Space d, ℝ)) :
    distDiv f η = ∑ i, distDeriv i f η i
```

lemma distDiv_ofFunction [source]

The divergence of a distribution from a bounded function.

```
lemma distDiv_ofFunction {dm1 : ℕ} {f : Space dm1.succ → EuclideanSpace ℝ (Fin dm1.succ)}
    {hf : IsDistBounded f} (η : 𝓢(Space dm1.succ, ℝ)) :
    distDiv (distOfFunction f hf) η =
    - ∫ x : Space dm1.succ, ⟪f x, Space.grad η x⟫_ℝ
```

2.30 PhysLean.SpaceAndTime.Space.Derivatives.Grad

PhysLean/SpaceAndTime/Space/Derivatives/Grad.lean on GitHub.

lemma ofLp_sum [source]

A lemma from Mathlib, which will be removed once in latest version of Mathlib.

```
@[simp]
lemma _root_.WithLp.ofLp_sum (p : ENNReal) (V : Type) [AddCommGroup V] [Module ℝ V] {ι : Type}
    (s : Finset ι) (f : ι → WithLp p V) : (∑ i ∈ s, f i).ofLp = ∑ i ∈ s, (f i).ofLp
```

lemma grad_inner_single [source]

```
lemma grad_inner_single {d} (f : Space d → ℝ) (x : Space d) (i : Fin d) :
    ⟪∇ f x, EuclideanSpace.single i 1⟫_ℝ = deriv i f x
```

lemma grad_inner_repr_eq [source]

```
lemma grad_inner_repr_eq {d} (f : Space d → ℝ) (x y : Space d) :
    ⟪∇ f x, (Space.basis).repr y⟫_ℝ = fderiv ℝ f x y
```

lemma repr_grad_inner_eq [source]

```
lemma repr_grad_inner_eq {d} (f : Space d → ℝ) (x y : Space d) :
    ⟪(Space.basis).repr x, ∇ f y⟫_ℝ = fderiv ℝ f y x
```

lemma gradient_eq_grad [source]

```
lemma gradient_eq_grad {d} (f : Space d → ℝ) :
    gradient f = basis.repr.symm ∘ ∇ f
```

lemma gradient_eq_sum [source]

```
lemma gradient_eq_sum {d} (f : Space d → ℝ) (x : Space d) :
    gradient f x = ∑ i, deriv i f x • basis i
```

lemma euclid_gradient_eq_sum [source]

```
lemma euclid_gradient_eq_sum {d} (f : EuclideanSpace ℝ (Fin d) → ℝ) (x : EuclideanSpace ℝ (Fin
    d)) :
    gradient f x = ∑ i, fderiv ℝ f x (EuclideanSpace.single i 1) • EuclideanSpace.single i 1
```

def distGrad [source]

The gradient of a distribution `(Space d) →d[ℝ] ℝ` as a distribution `(Space d) →d[ℝ] (EuclideanSpace ℝ (Fin d))`.

```
noncomputable def distGrad {d} :
    ((Space d) →d[ℝ] ℝ) →ₗ[ℝ] (Space d) →d[ℝ] (EuclideanSpace ℝ (Fin d)) where
```

lemma distGrad_inner_eq [source]

```
lemma distGrad_inner_eq {d} (f : (Space d) →d[ℝ] ℝ) (η : 𝓢(Space d, ℝ))
    (y : EuclideanSpace ℝ (Fin d)) : ⟪distGrad f η, y⟫_ℝ = fderivD ℝ f η (basis.repr.symm y)
```

lemma distGrad_eq_of_inner [source]

```
lemma distGrad_eq_of_inner {d} (f : (Space d) →d[ℝ] ℝ)
    (g : (Space d) →d[ℝ] EuclideanSpace ℝ (Fin d))
    (h : ∀ η y, fderivD ℝ f η y = ⟪g η, basis.repr y⟫_ℝ) :
    distGrad f = g
```

lemma distGrad_eq_sum_basis [source]

```
lemma distGrad_eq_sum_basis {d} (f : (Space d) →d[ℝ] ℝ) (η : 𝓢(Space d, ℝ)) :
    distGrad f η = ∑ i, - f (SchwartzMap.evalCLM (𝕜 := ℝ) (basis i) (fderivCLM ℝ η)) •
      EuclideanSpace.single i 1
```

lemma distGrad_toFun_eq_distDeriv [source]

```
lemma distGrad_toFun_eq_distDeriv {d} (f : (Space d) →d[ℝ] ℝ) :
    (distGrad f).toFun = fun ε => WithLp.toLp 2 fun i => distDeriv i f ε
```

lemma distGrad_apply [source]

```
lemma distGrad_apply {d} (f : (Space d) →d[ℝ] ℝ) (ε : 𝓢(Space d, ℝ)) :
    (distGrad f) ε = fun i => distDeriv i f ε
```

2.31 PhysLean.SpaceAndTime.Space.DistConst

PhysLean/SpaceAndTime/Space/DistConst.lean on GitHub.

def distConst [source]

The constant distribution from `Space d` to a module `M` associated with `m : M`.

```
noncomputable def distConst {M } [NormedAddCommGroup M] [NormedSpace ℝ M] (d : ℕ) (m : M) :
    (Space d) →d[ℝ] M
```

lemma distDeriv_distConst [source]

```
@[simp]
lemma distDeriv_distConst {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (μ : Fin d) (m : M) :
    distDeriv μ (distConst d m) = 0
```

lemma distGrad_distConst [source]

```
@[simp]
lemma distGrad_distConst {d} (m : ℝ) :
    distGrad (distConst d m) = 0
```

lemma distDiv_distConst [source]

```
@[simp]
lemma distDiv_distConst {d} (m : EuclideanSpace ℝ (Fin d)) :
    distDiv (distConst d m) = 0
```

lemma distCurl_distConst [source]

```
@[simp]
lemma distCurl_distConst (m : EuclideanSpace ℝ (Fin 3)) :
    distCurl (distConst 3 m) = 0
```

2.32 PhysLean.SpaceAndTime.Space.DistOfFunction

PhysLean/SpaceAndTime/Space/DistOfFunction.lean on GitHub.

def distOfFunction [source]

A distribution `Space d →d[ℝ] F` from a function `f : Space d → F` which satisfies the `IsDistBounded f` condition.

```
def distOfFunction {d : ℕ} (f : Space d → F) (hf : IsDistBounded f) :
    (Space d) →d[ℝ] F
```

lemma distOfFunction_apply [source]

```
lemma distOfFunction_apply {d : ℕ} (f : Space d → F)
    (hf : IsDistBounded f) (η : 𝓢(Space d, ℝ)) :
    distOfFunction f hf η = ∫ x, η x • f x
```

lemma distOfFunction_zero_eq_zero [source]

```
@[simp]
lemma distOfFunction_zero_eq_zero {d : ℕ} :
    distOfFunction (fun _ : Space d => (0 : F)) (by fun_prop) = 0
```

lemma distOfFunction_smul [source]

```
lemma distOfFunction_smul {d : ℕ} (f : Space d → F)
    (hf : IsDistBounded f) (c : ℝ) :
    distOfFunction (c • f) (by fun_prop) = c • distOfFunction f hf
```

lemma distOfFunction_smul_fun [source]

```
lemma distOfFunction_smul_fun {d : ℕ} (f : Space d → F)
    (hf : IsDistBounded f) (c : ℝ) :
    distOfFunction (fun x => c • f x) (by fun_prop) = c • distOfFunction f hf
```

lemma distOfFunction_mul_fun [source]

```
lemma distOfFunction_mul_fun {d : ℕ} (f : Space d → ℝ)
    (hf : IsDistBounded f) (c : ℝ) :
    distOfFunction (fun x => c * f x) (by fun_prop) = c • distOfFunction f hf
```

lemma distOfFunction_neg [source]

```
lemma distOfFunction_neg {d : ℕ} (f : Space d → F)
    (hf : IsDistBounded (fun x => - f x)) :
    distOfFunction (fun x => - f x) hf = - distOfFunction f (by simpa using hf.neg)
```

lemma distOfFunction_inner [source]

```
lemma distOfFunction_inner {d n : ℕ} (f : Space d → EuclideanSpace ℝ (Fin n))
    (hf : IsDistBounded f)
    (η : 𝓢(Space d, ℝ)) (y : EuclideanSpace ℝ (Fin n)) :
    ⟪distOfFunction f hf η, y⟫_ℝ = ∫ x, η x * ⟪f x, y⟫_ℝ
```

lemma distOfFunction_eculid_eval [source]

```
lemma distOfFunction_eculid_eval {d n : ℕ} (f : Space d → EuclideanSpace ℝ (Fin n))
    (hf : IsDistBounded f) (η : 𝓢(Space d, ℝ)) (i : Fin n) :
    distOfFunction f hf η i = distOfFunction (fun x => f x i) (hf.pi_comp i) η
```

lemma distOfFunction_vector_eval [source]

```
lemma distOfFunction_vector_eval {d : ℕ} (f : Space d → Lorentz.Vector d)
    (hf : IsDistBounded f) (η : 𝓢(Space d, ℝ)) (i : Fin 1 ⊕ Fin d) :
    distOfFunction f hf η i = distOfFunction (fun x => f x i) (hf.vector_component i) η
```

2.33 PhysLean.SpaceAndTime.Space.IsDistBounded

PhysLean/SpaceAndTime/Space/IsDistBounded.lean on GitHub.

def IsDistBounded [source]

The boundedness condition on a function `EuclideanSpace ℝ (Fin dm1.succ) → F` for it to form a distribution.

```
@[fun_prop]
def IsDistBounded {d : ℕ} (f : Space d → F) : Prop
```

lemma aestronglyMeasurable [source]

```
@[fun_prop]
lemma aestronglyMeasurable {d : ℕ} {f : Space d → F} (hf : IsDistBounded f) :
    AEStronglyMeasurable (fun x => f x) volume
```

lemma aeStronglyMeasurable_schwartzMap_smul [source]

```
@[fun_prop]
lemma aeStronglyMeasurable_schwartzMap_smul {d : ℕ} {f : Space d → F}
    (hf : IsDistBounded f) (η : 𝓢(Space d, ℝ)) :
    AEStronglyMeasurable (fun x => η x • f x)
```

lemma aeStronglyMeasurable_fderiv_schwartzMap_smul [source]

```
@[fun_prop]
lemma aeStronglyMeasurable_fderiv_schwartzMap_smul {d : ℕ} {f : Space d → F}
    (hf : IsDistBounded f) (η : 𝓢(Space d, ℝ)) (y : Space d) :
    AEStronglyMeasurable (fun x => fderiv ℝ η x y • f x)
```

lemma aeStronglyMeasurable_inv_pow [source]

```
@[fun_prop]
lemma aeStronglyMeasurable_inv_pow {d r : ℕ} {f : Space d → F}
    (hf : IsDistBounded f) :
    AEStronglyMeasurable (fun x => ‖((1 + ‖x‖) ^ r)⁻¹‖ • f x)
```

lemma aeStronglyMeasurable_time_schwartzMap_smul [source]

```
@[fun_prop]
lemma aeStronglyMeasurable_time_schwartzMap_smul {d : ℕ} {f : Space d → F}
    (hf : IsDistBounded f) (η : 𝓢(Time × Space d, ℝ)) :
    AEStronglyMeasurable (fun x => η x • f x.2)
```

lemma integrable_space [source]

```
@[fun_prop]
lemma integrable_space {d : ℕ} {f : Space d → F} (hf : IsDistBounded f)
    (η : 𝓢(Space d, ℝ)) :
    Integrable (fun x : Space d => η x • f x) volume
```

lemma integrable_space_mul [source]

```
@[fun_prop]
lemma integrable_space_mul {d : ℕ} {f : Space d → ℝ} (hf : IsDistBounded f)
    (η : 𝓢(Space d, ℝ)) :
    Integrable (fun x : Space d => η x * f x) volume
```

lemma integrable_space_fderiv [source]

```
@[fun_prop]
lemma integrable_space_fderiv {d : ℕ} {f : Space d → F} (hf : IsDistBounded f)
    (η : 𝓢(Space d, ℝ)) (y : Space d) :
    Integrable (fun x : Space d => fderiv ℝ η x y • f x) volume
```

lemma integrable_space_fderiv_mul [source]

```
@[fun_prop]
lemma integrable_space_fderiv_mul {d : ℕ} {f : Space d → ℝ} (hf : IsDistBounded f)
    (η : 𝓢(Space d, ℝ)) (y : Space d) :
    Integrable (fun x : Space d => fderiv ℝ η x y * f x) volume
```

instance Measure.HasTemperateGrowth (μ1.prod μ2) [source]

```
instance {D1 : Type} [NormedAddCommGroup D1] [MeasurableSpace D1]
    {D2 : Type} [NormedAddCommGroup D2] [MeasurableSpace D2]
    (μ1 : Measure D1) (μ2 : Measure D2)
    [Measure.HasTemperateGrowth μ1] [Measure.HasTemperateGrowth μ2]
    [OpensMeasurableSpace (D1 × D2)] :
    Measure.HasTemperateGrowth (μ1.prod μ2) where
```

lemma integrable_time_space [source]

```
@[fun_prop]
lemma integrable_time_space {d : ℕ} {f : Space d → F} (hf : IsDistBounded f)
    (η : 𝓢(Time × Space d, ℝ)) :
    Integrable (fun x : Time × Space d => η x • f x.2) volume
```

lemma integrable_mul_inv_pow [source]

```
lemma integrable_mul_inv_pow {d : ℕ}
    {f : Space d → F} (hf : IsDistBounded f) :
    ∃ r, Integrable (fun x => ‖((1 + ‖x‖) ^ r)⁻¹‖ • f x) volume
```

lemma integral_mul_schwartzMap_bounded [source]

```
lemma integral_mul_schwartzMap_bounded {d : ℕ} {f : Space d → F} (hf : IsDistBounded f) :
    ∃ (s : Finset (ℕ × ℕ)) (C : ℝ),
    0 ≤ C ∧ ∀ (η : 𝓢(Space d, ℝ)),
    ‖∫ (x : Space d), η x • f x‖ ≤ C * (s.sup (schwartzSeminormFamily ℝ (Space d) ℝ)) η
```

lemma zero [source]

```
@[fun_prop]
lemma zero {d} : IsDistBounded (0 : Space d → F)
```

lemma add [source]

```
@[fun_prop]
lemma add {d : ℕ} {f g : Space d → F}
    (hf : IsDistBounded f) (hg : IsDistBounded g) : IsDistBounded (f + g)
```

lemma fun_add [source]

```
@[fun_prop]
lemma fun_add {d : ℕ} {f g : Space d → F}
    (hf : IsDistBounded f) (hg : IsDistBounded g) : IsDistBounded (fun x => f x + g x)
```

lemma sum [source]

```
lemma sum {ι : Type*} {s : Finset ι} {d : ℕ} {f : ι → Space d → F}
    (hf : ∀ i ∈ s, IsDistBounded (f i)) : IsDistBounded (∑ i ∈ s, f i)
```

lemma sum_fun [source]

```
lemma sum_fun {ι : Type*} {s : Finset ι} {d : ℕ} {f : ι → Space d → F}
    (hf : ∀ i ∈ s, IsDistBounded (f i)) : IsDistBounded (fun x => ∑ i ∈ s, f i x)
```

lemma const_smul [source]

```
@[fun_prop]
lemma const_smul {d : ℕ} [NormedSpace ℝ F] {f : Space d → F}
    (hf : IsDistBounded f) (c : ℝ) : IsDistBounded (c • f)
```

lemma neg [source]

```
@[fun_prop]
lemma neg {d : ℕ} [NormedSpace ℝ F] {f : Space d → F}
    (hf : IsDistBounded f) : IsDistBounded (fun x => - f x)
```

lemma const_fun_smul [source]

```
@[fun_prop]
lemma const_fun_smul {d : ℕ} [NormedSpace ℝ F] {f : Space d → F}
    (hf : IsDistBounded f) (c : ℝ) : IsDistBounded (fun x => c • f x)
```

lemma const_mul_fun [source]

```
@[fun_prop]
lemma const_mul_fun {d : ℕ}
    {f : Space d → ℝ}
    (hf : IsDistBounded f) (c : ℝ) : IsDistBounded (fun x => c * f x)
```

lemma mul_const_fun [source]

```
@[fun_prop]
lemma mul_const_fun {d : ℕ}
    {f : Space d → ℝ}
    (hf : IsDistBounded f) (c : ℝ) : IsDistBounded (fun x => f x * c)
```

lemma pi_comp [source]

```
@[fun_prop]
lemma pi_comp {d n : ℕ}
    {f : Space d → EuclideanSpace ℝ (Fin n)}
    (hf : IsDistBounded f) (j : Fin n) : IsDistBounded (fun x => f x j)
```

lemma vector_component [source]

```
lemma vector_component {d: ℕ} {f : Space d → Lorentz.Vector d}
    (hf : IsDistBounded f) (j : Fin 1 ⊕ Fin d) : IsDistBounded (fun x => f x j)
```

lemma comp_add_right [source]

```
lemma comp_add_right {d : ℕ} {f : Space d → F}
    (hf : IsDistBounded f) (c : Space d) :
    IsDistBounded (fun x => f (x + c))
```

lemma comp_sub_right [source]

```
lemma comp_sub_right {d : ℕ} {f : Space d → F}
    (hf : IsDistBounded f) (c : Space d) :
    IsDistBounded (fun x => f (x - c))
```

lemma congr [source]

```
lemma congr {d : ℕ} {f : Space d → F}
    {g : Space d → F'}
    (hf : IsDistBounded f) (hae : AEStronglyMeasurable g) (hfg : ∀ x, ‖g x‖ = ‖f x‖) :
      IsDistBounded g
```

lemma mono [source]

```
lemma mono {d : ℕ} {f : Space d → F}
    {g : Space d → F'}
    (hf : IsDistBounded f) (hae : AEStronglyMeasurable g)
    (hfg : ∀ x, ‖g x‖ ≤ ‖f x‖) : IsDistBounded g
```

lemma inner_left [source]

```
@[fun_prop]
lemma inner_left {d n : ℕ}
    {f : Space d → EuclideanSpace ℝ (Fin n) }
    (hf : IsDistBounded f) (y : EuclideanSpace ℝ (Fin n)) :
    IsDistBounded (fun x => ⟪f x, y⟫_ℝ)
```

lemma smul_const [source]

```
@[fun_prop]
lemma smul_const {d : ℕ} [NormedSpace ℝ F] {c : Space d → ℝ}
    (hc : IsDistBounded c) (f : F) : IsDistBounded (fun x => c x • f)
```

lemma const [source]

```
@[fun_prop]
lemma const {d : ℕ} (f : F) :
    IsDistBounded (d := d) (fun _ : Space d => f)
```

lemma pow [source]

```
@[fun_prop]
lemma pow {d : ℕ} (n : ℤ) (hn : - (d - 1 : ℕ) ≤ n) :
    IsDistBounded (d := d) (fun x => ‖x‖ ^ n)
```

lemma pow_shift [source]

```
@[fun_prop]
lemma pow_shift {d : ℕ} (n : ℤ)
    (g : Space d) (hn : - (d - 1 : ℕ) ≤ n) :
    IsDistBounded (d := d) (fun x => ‖x - g‖ ^ n)
```

lemma nat_pow [source]

```
@[fun_prop]
lemma nat_pow {d : ℕ} (n : ℕ) :
    IsDistBounded (d := d) (fun x => ‖x‖ ^ n)
```

lemma norm_add_nat_pow [source]

```
@[fun_prop]
lemma norm_add_nat_pow {d : ℕ} (n : ℕ) (a : ℝ) :
    IsDistBounded (d := d) (fun x => (‖x‖ + a) ^ n)
```

lemma norm_add_pos_nat_zpow [source]

```
@[fun_prop]
lemma norm_add_pos_nat_zpow {d : ℕ} (n : ℤ) (a : ℝ) (ha : 0 < a) :
    IsDistBounded (d := d) (fun x => (‖x‖ + a) ^ n)
```

lemma nat_pow_shift [source]

```
@[fun_prop]
lemma nat_pow_shift {d : ℕ} (n : ℕ)
    (g : Space d) :
    IsDistBounded (d := d) (fun x => ‖x - g‖ ^ n)
```

lemma inv [source]

```
@[fun_prop]
lemma inv {n : ℕ} :
    IsDistBounded (d := n.succ.succ) (fun x => ‖x‖⁻¹)
```

lemma norm [source]

```
@[fun_prop]
lemma norm {d : ℕ} : IsDistBounded (d := d) (fun x => ‖x‖)
```

lemma log_norm [source]

```
@[fun_prop]
lemma log_norm {d : ℕ} :
    IsDistBounded (d := d.succ.succ) (fun x => Real.log ‖x‖)
```

lemma zpow_smul_self [source]

```
lemma zpow_smul_self {d : ℕ} (n : ℤ) (hn : - (d - 1 : ℕ) - 1 ≤ n) :
    IsDistBounded (d := d) (fun x => ‖x‖ ^ n • x)
```

lemma zpow_smul_repr_self [source]

```
lemma zpow_smul_repr_self {d : ℕ} (n : ℤ) (hn : - (d - 1 : ℕ) - 1 ≤ n) :
    IsDistBounded (d := d) (fun x => ‖x‖ ^ n • basis.repr x)
```

lemma inv_pow_smul_self [source]

```
lemma inv_pow_smul_self {d : ℕ} (n : ℕ) (hn : - (d - 1 : ℕ) - 1 ≤ (- n : ℤ)) :
    IsDistBounded (d := d) (fun x => ‖x‖⁻¹ ^ n • x)
```

lemma inv_pow_smul_repr_self [source]

```
lemma inv_pow_smul_repr_self {d : ℕ} (n : ℕ) (hn : - (d - 1 : ℕ) - 1 ≤ (- n : ℤ)) :
    IsDistBounded (d := d) (fun x => ‖x‖⁻¹ ^ n • basis.repr x)
```

lemma norm_smul_nat_pow [source]

```
lemma norm_smul_nat_pow {d} (p : ℕ) (c : Space d) :
    IsDistBounded (fun x => ‖x‖ * ‖x + c‖ ^ p)
```

lemma norm_smul_zpow [source]

```
lemma norm_smul_zpow {d} (p : ℤ) (c : Space d) (hn : - (d - 1 : ℕ) ≤ p) :
    IsDistBounded (fun x => ‖x‖ * ‖x + c‖ ^ p)
```

lemma norm_smul_isDistBounded [source]

```
@[fun_prop]
lemma norm_smul_isDistBounded {d : ℕ} [NormedSpace ℝ F] {f : Space d → F}
    (hf : IsDistBounded f) :
    IsDistBounded (fun x => ‖x‖ • f x)
```

lemma norm_mul_isDistBounded [source]

```
@[fun_prop]
lemma norm_mul_isDistBounded {d : ℕ} {f : Space d → ℝ}
    (hf : IsDistBounded f) :
    IsDistBounded (fun x => ‖x‖ * f x)
```

lemma component_smul_isDistBounded [source]

```
@[fun_prop]
lemma component_smul_isDistBounded {d : ℕ} [NormedSpace ℝ F] {f : Space d → F}
    (hf : IsDistBounded f) (i : Fin d) :
    IsDistBounded (fun x => x i • f x)
```

lemma component_mul_isDistBounded [source]

```
@[fun_prop]
lemma component_mul_isDistBounded {d : ℕ} {f : Space d → ℝ}
    (hf : IsDistBounded f) (i : Fin d) :
    IsDistBounded (fun x => x i * f x)
```

lemma isDistBounded_smul_self [source]

```
@[fun_prop]
lemma isDistBounded_smul_self {d : ℕ} {f : Space d → ℝ}
    (hf : IsDistBounded f) : IsDistBounded (fun x => f x • x)
```

lemma isDistBounded_smul_self_repr [source]

```
@[fun_prop]
lemma isDistBounded_smul_self_repr {d : ℕ} {f : Space d → ℝ}
    (hf : IsDistBounded f) : IsDistBounded (fun x => f x • basis.repr x)
```

lemma isDistBounded_smul_inner [source]

```
@[fun_prop]
lemma isDistBounded_smul_inner {d : ℕ} [NormedSpace ℝ F] {f : Space d → F}
    (hf : IsDistBounded f) (y : Space d) : IsDistBounded (fun x => ⟪y, x⟫_ℝ • f x)
```

lemma isDistBounded_smul_inner_of_smul_norm [source]

```
lemma isDistBounded_smul_inner_of_smul_norm {d : ℕ} [NormedSpace ℝ F] {f : Space d → F}
    (hf : IsDistBounded (fun x => ‖x‖ • f x)) (hae : AEStronglyMeasurable f) (y : Space d) :
    IsDistBounded (fun x => ⟪y, x⟫_ℝ • f x)
```

lemma isDistBounded_mul_inner [source]

```
@[fun_prop]
lemma isDistBounded_mul_inner {d : ℕ} {f : Space d → ℝ}
    (hf : IsDistBounded f) (y : Space d) : IsDistBounded (fun x => ⟪y, x⟫_ℝ * f x)
```

lemma isDistBounded_mul_inner' [source]

```
lemma isDistBounded_mul_inner' {d : ℕ} {f : Space d → ℝ}
    (hf : IsDistBounded f) (y : Space d) : IsDistBounded (fun x => ⟪x, y⟫_ℝ * f x)
```

lemma isDistBounded_mul_inner_of_smul_norm [source]

```
lemma isDistBounded_mul_inner_of_smul_norm {d : ℕ} {f : Space d → ℝ}
    (hf : IsDistBounded (fun x => ‖x‖ * f x)) (hae : AEStronglyMeasurable f) (y : Space d) :
    IsDistBounded (fun x => ⟪y, x⟫_ℝ * f x)
```

lemma mul_inner_pow_neg_two [source]

```
@[fun_prop]
lemma mul_inner_pow_neg_two {d : ℕ}
    (y : Space d.succ.succ) :
    IsDistBounded (fun x => ⟪y, x⟫_ℝ * ‖x‖ ^ (- 2 : ℤ))
```

2.34 PhysLean.SpaceAndTime.Space.Norm

PhysLean/SpaceAndTime/Space/Norm.lean on GitHub.

def normPowerSeries [source]

A power series which is differentiable everywhere, and in the limit as $n \to \infty$ tends to $\|x\|$.

```
def normPowerSeries {d} : ℕ → Space d → ℝ
```

lemma normPowerSeries_eq [source]

```
lemma normPowerSeries_eq (n : ℕ) :
    normPowerSeries (d := d) n = fun x => √(‖x‖ ^ 2 + 1/(n + 1))
```

lemma normPowerSeries_eq_rpow [source]

```
lemma normPowerSeries_eq_rpow {d} (n : ℕ) :
    normPowerSeries (d := d) n = fun x => ((‖x‖ ^ 2 + 1/(n + 1))) ^ (1/2 : ℝ)
```

lemma normPowerSeries_differentiable [source]

```
@[fun_prop]
lemma normPowerSeries_differentiable {d} (n : ℕ) :
    Differentiable ℝ (fun (x : Space d) => normPowerSeries n x)
```

lemma normPowerSeries_tendsto [source]

```
lemma normPowerSeries_tendsto {d} (x : Space d) (hx : x ≠ 0) :
    Filter.Tendsto (fun n => normPowerSeries n x) Filter.atTop (𝓝 (‖x‖))
```

lemma normPowerSeries_inv_tendsto [source]

```
lemma normPowerSeries_inv_tendsto {d} (x : Space d) (hx : x ≠ 0) :
    Filter.Tendsto (fun n => (normPowerSeries n x)⁻¹) Filter.atTop (𝓝 (‖x‖⁻¹))
```

lemma deriv_normPowerSeries [source]

```
lemma deriv_normPowerSeries {d} (n : ℕ) (x : Space d) (i : Fin d) :
    ∂[i] (normPowerSeries n) x = x i * (normPowerSeries n x)⁻¹
```

lemma fderiv_normPowerSeries [source]

```
lemma fderiv_normPowerSeries {d} (n : ℕ) (x y : Space d) :
    fderiv ℝ (fun (x : Space d) => normPowerSeries n x) x y =
      ⟪y, x⟫_ℝ * (normPowerSeries n x)⁻¹
```

lemma deriv_normPowerSeries_tendsto [source]

```
lemma deriv_normPowerSeries_tendsto {d} (x : Space d) (hx : x ≠ 0) (i : Fin d) :
    Filter.Tendsto (fun n => ∂[i] (normPowerSeries n) x) Filter.atTop (𝓝 (x i * (‖x‖)⁻¹))
```

lemma fderiv_normPowerSeries_tendsto [source]

```
lemma fderiv_normPowerSeries_tendsto {d} (x y : Space d) (hx : x ≠ 0) :
    Filter.Tendsto (fun n => fderiv ℝ (fun (x : Space d) => normPowerSeries n x) x y)
      Filter.atTop (𝓝 (⟪y, x⟫_ℝ * (‖x‖)⁻¹))
```

lemma normPowerSeries_aestronglyMeasurable [source]

```
@[fun_prop]
lemma normPowerSeries_aestronglyMeasurable {d} (n : ℕ) :
    AEStronglyMeasurable (normPowerSeries n : Space d → ℝ) volume
```

lemma normPowerSeries_nonneg [source]

```
@[simp]
lemma normPowerSeries_nonneg {d} (n : ℕ) (x : Space d) :
    0 ≤ normPowerSeries n x
```

lemma normPowerSeries_pos [source]

```
@[simp]
lemma normPowerSeries_pos {d} (n : ℕ) (x : Space d) :
    0 < normPowerSeries n x
```

lemma normPowerSeries_ne_zero [source]

```
@[simp]
lemma normPowerSeries_ne_zero {d} (n : ℕ) (x : Space d) :
    normPowerSeries n x ≠ 0
```

lemma normPowerSeries_le_norm_sq_add_one [source]

```
lemma normPowerSeries_le_norm_sq_add_one {d} (n : ℕ) (x : Space d) :
    normPowerSeries n x ≤ ‖x‖ + 1
```

lemma norm_lt_normPowerSeries [source]

```
@[simp]
lemma norm_lt_normPowerSeries {d} (n : ℕ) (x : Space d) :
    ‖x‖ < normPowerSeries n x
```

lemma norm_le_normPowerSeries [source]

```
lemma norm_le_normPowerSeries {d} (n : ℕ) (x : Space d) :
    ‖x‖ ≤ normPowerSeries n x
```

lemma normPowerSeries_zpow_le_norm_sq_add_one [source]

```
lemma normPowerSeries_zpow_le_norm_sq_add_one {d} (n : ℕ) (m : ℤ) (x : Space d)
    (hx : x ≠ 0) :
    (normPowerSeries n x) ^ m ≤ (‖x‖ + 1) ^ m + ‖x‖ ^ m
```

lemma normPowerSeries_inv_le [source]

```
lemma normPowerSeries_inv_le {d} (n : ℕ) (x : Space d) (hx : x ≠ 0) :
    (normPowerSeries n x)⁻¹ ≤ ‖x‖⁻¹
```

lemma normPowerSeries_log_le_normPowerSeries [source]

```
lemma normPowerSeries_log_le_normPowerSeries {d} (n : ℕ) (x : Space d) :
    |Real.log (normPowerSeries n x)| ≤ (normPowerSeries n x)⁻¹ + (normPowerSeries n x)
```

lemma normPowerSeries_log_le [source]

```
lemma normPowerSeries_log_le {d} (n : ℕ) (x : Space d) (hx : x ≠ 0) :
    |Real.log (normPowerSeries n x)| ≤ ‖x‖⁻¹ + (‖x‖ + 1)
```

lemma normPowerSeries_zpow [source]

```
@[fun_prop]
lemma IsDistBounded.normPowerSeries_zpow {d : ℕ} {n : ℕ} (m : ℤ) :
    IsDistBounded (d := d) (fun x => (normPowerSeries n x) ^ m)
```

lemma normPowerSeries_single [source]

```
@[fun_prop]
lemma IsDistBounded.normPowerSeries_single {d : ℕ} {n : ℕ} :
    IsDistBounded (d := d) (fun x => (normPowerSeries n x))
```

lemma normPowerSeries_inv [source]

```
@[fun_prop]
lemma IsDistBounded.normPowerSeries_inv {d : ℕ} {n : ℕ} :
    IsDistBounded (d := d) (fun x => (normPowerSeries n x)⁻¹)
```

lemma normPowerSeries_deriv [source]

```
@[fun_prop]
lemma IsDistBounded.normPowerSeries_deriv {d : ℕ} (n : ℕ) (i : Fin d) :
    IsDistBounded (d := d) (fun x => ∂[i] (normPowerSeries n) x)
```

lemma normPowerSeries_fderiv [source]

```
@[fun_prop]
lemma IsDistBounded.normPowerSeries_fderiv {d : ℕ} (n : ℕ) (y : Space d) :
    IsDistBounded (d := d) (fun x => fderiv ℝ (fun (x : Space d) => normPowerSeries n x) x y)
```

lemma normPowerSeries_log [source]

```
@[fun_prop]
lemma IsDistBounded.normPowerSeries_log {d : ℕ} (n : ℕ) :
    IsDistBounded (d := d) (fun x => Real.log (normPowerSeries n x))
```

lemma differentiable_normPowerSeries_zpow [source]

```
@[fun_prop]
lemma differentiable_normPowerSeries_zpow {d : ℕ} {n : ℕ} (m : ℤ) :
    Differentiable ℝ (fun x : Space d => (normPowerSeries n x) ^ m)
```

lemma differentiable_normPowerSeries_inv [source]

```
@[fun_prop]
lemma differentiable_normPowerSeries_inv {d : ℕ} {n : ℕ} :
    Differentiable ℝ (fun x : Space d => (normPowerSeries n x)⁻¹)
```

lemma differentiable_log_normPowerSeries [source]

```
@[fun_prop]
lemma differentiable_log_normPowerSeries {d : ℕ} {n : ℕ} :
```

```
    Differentiable ℝ (fun x : Space d => Real.log (normPowerSeries n x))
```

lemma deriv_normPowerSeries_zpow [source]

```
lemma deriv_normPowerSeries_zpow {d : ℕ} {n : ℕ} (m : ℤ) (x : Space d) (i : Fin d) :
    ∂[i] (fun x : Space d => (normPowerSeries n x) ^ m) x =
      m * x i * (normPowerSeries n x) ^ (m - 2)
```

lemma fderiv_normPowerSeries_zpow [source]

```
lemma fderiv_normPowerSeries_zpow {d : ℕ} {n : ℕ} (m : ℤ) (x y : Space d) :
    fderiv ℝ (fun x : Space d => (normPowerSeries n x) ^ m) x y =
      m * ⟪y, x⟫_ℝ * (normPowerSeries n x) ^ (m - 2)
```

lemma deriv_log_normPowerSeries [source]

```
lemma deriv_log_normPowerSeries {d : ℕ} {n : ℕ} (x : Space d) (i : Fin d) :
    ∂[i] (fun x : Space d => Real.log (normPowerSeries n x)) x =
      x i * (normPowerSeries n x) ^ (-2 : ℤ)
```

lemma fderiv_log_normPowerSeries [source]

```
lemma fderiv_log_normPowerSeries {d : ℕ} {n : ℕ} (x y : Space d) :
    fderiv ℝ (fun x : Space d => Real.log (normPowerSeries n x)) x y =
      ⟪y, x⟫_ℝ * (normPowerSeries n x) ^ (-2 : ℤ)
```

lemma gradient_dist_normPowerSeries_zpow [source]

```
lemma gradient_dist_normPowerSeries_zpow {d : ℕ} {n : ℕ} (m : ℤ) :
    distGrad (distOfFunction (fun x : Space d => (normPowerSeries n x) ^ m) (by fun_prop)) =
    distOfFunction (fun x : Space d => (m * (normPowerSeries n x) ^ (m - 2)) • basis.repr x)
    (by fun_prop)
```

lemma gradient_dist_normPowerSeries_zpow_tendsTo_distGrad_norm [source]

```
lemma gradient_dist_normPowerSeries_zpow_tendsTo_distGrad_norm {d : ℕ} (m : ℤ)
    (hm : - (d.succ - 1 : ℕ) ≤ m) (η : 𝓢(Space d.succ, ℝ))
    (y : EuclideanSpace ℝ (Fin d.succ)) :
    Filter.Tendsto (fun n =>
    ⟪(distGrad (distOfFunction
    (fun x : Space d.succ => (normPowerSeries n x) ^ m) (by fun_prop))) η, y⟫_ℝ)
    Filter.atTop
    (𝓝 (⟪distGrad (distOfFunction (fun x : Space d.succ => ‖x‖ ^ m)
    (IsDistBounded.pow m hm)) η, y⟫_ℝ))
```

lemma gradient_dist_normPowerSeries_zpow_tendsTo [source]

```
lemma gradient_dist_normPowerSeries_zpow_tendsTo {d : ℕ} (m : ℤ) (hm : - (d.succ - 1 : ℕ) + 1
    ≤ m)
    (η : 𝓢(Space d.succ, ℝ)) (y : EuclideanSpace ℝ (Fin d.succ)) :
    Filter.Tendsto (fun n =>
    ⟪(distGrad (distOfFunction (fun x : Space d.succ => (normPowerSeries n x) ^ m)
    (by fun_prop))) η, y⟫_ℝ)
```

```
    Filter.atTop
    (𝒩 (⟪distOfFunction (fun x : Space d.succ => (m * ‖x‖ ^ (m - 2)) • basis.repr x) (by
    simp [← smul_smul]
    refine IsDistBounded.const_fun_smul ?_ ↑m
    apply IsDistBounded.zpow_smul_repr_self
    simp_all
    grind) η, y⟫_ℝ))
```

lemma gradient_dist_normPowerSeries_log [source]

```
lemma gradient_dist_normPowerSeries_log {d : ℕ} {n : ℕ} :
    distGrad (distOfFunction (fun x : Space d => Real.log (normPowerSeries n x)) (by fun_prop))
     =
    distOfFunction (fun x : Space d => ((normPowerSeries n x) ^ (- 2 : ℤ)) • basis.repr x)
    (by fun_prop)
```

lemma gradient_dist_normPowerSeries_log_tendsTo_distGrad_norm [source]

```
lemma gradient_dist_normPowerSeries_log_tendsTo_distGrad_norm {d : ℕ}
    (η : 𝒮(Space d.succ.succ, ℝ)) (y : EuclideanSpace ℝ (Fin d.succ.succ)) :
    Filter.Tendsto (fun n =>
    ⟪(distGrad (distOfFunction
    (fun x : Space d.succ.succ => Real.log (normPowerSeries n x)) (by fun_prop))) η, y⟫_ℝ)
    Filter.atTop
    (𝒩 (⟪distGrad (distOfFunction (fun x : Space d.succ.succ => Real.log ‖x‖)
    (IsDistBounded.log_norm)) η, y⟫_ℝ))
```

lemma gradient_dist_normPowerSeries_log_tendsTo [source]

```
lemma gradient_dist_normPowerSeries_log_tendsTo {d : ℕ}
    (η : 𝒮(Space d.succ.succ, ℝ)) (y : EuclideanSpace ℝ (Fin d.succ.succ)) :
    Filter.Tendsto (fun n =>
    ⟪(distGrad (distOfFunction (fun x : Space d.succ.succ => Real.log (normPowerSeries n x))
    (by fun_prop))) η, y⟫_ℝ)
    Filter.atTop
    (𝒩 (⟪distOfFunction (fun x : Space d.succ.succ => (‖x‖ ^ (- 2 : ℤ)) • basis.repr x) (by
    refine (IsDistBounded.zpow_smul_repr_self _ ?_)
    simp_all) η, y⟫_ℝ))
```

lemma distGrad_distOfFunction_norm_zpow [source]

```
lemma distGrad_distOfFunction_norm_zpow {d : ℕ} (m : ℤ) (hm : - (d.succ - 1 : ℕ) + 1 ≤ m) :
    distGrad (distOfFunction (fun x : Space d.succ => ‖x‖ ^ m)
      (IsDistBounded.pow m (by simp_all; omega)))
    = distOfFunction (fun x : Space d.succ => (m * ‖x‖ ^ (m - 2)) • basis.repr x) (by
      simp [← smul_smul]
      refine IsDistBounded.const_fun_smul ?_ ↑m
      apply IsDistBounded.zpow_smul_repr_self
      simp_all
      omega)
```

lemma distGrad_distOfFunction_log_norm [source]

```
lemma distGrad_distOfFunction_log_norm {d : ℕ} :
```

```
    distGrad (distOfFunction (fun x : Space d.succ.succ => Real.log ‖x‖)
      (IsDistBounded.log_norm))
    = distOfFunction (fun x : Space d.succ.succ => (‖x‖ ^ (- 2 : ℤ)) • basis.repr x) (by
      refine (IsDistBounded.zpow_smul_repr_self _ ?_)
      simp_all)
```

lemma distDiv_inv_pow_eq_dim [source]

```
lemma distDiv_inv_pow_eq_dim {d : ℕ} :
    distDiv (distOfFunction (fun x : Space d.succ => ‖x‖ ^ (- d.succ : ℤ) • basis.repr x)
      (IsDistBounded.zpow_smul_repr_self (- d.succ : ℤ) (by omega))) =
      (d.succ * (volume (α := Space d.succ)).real (Metric.ball 0 1)) • diracDelta ℝ 0
```

2.35 PhysLean.SpaceAndTime.Space.RadialAngularMeasure

PhysLean/SpaceAndTime/Space/RadialAngularMeasure.lean on GitHub.

def radialAngularMeasure [source]

The measure on `Space d` weighted by `1 / ‖x‖ ^ (d - 1)`.

```
def radialAngularMeasure {d : ℕ} : Measure (Space d)
```

lemma radialAngularMeasure_eq_volume_withDensity [source]

```
lemma radialAngularMeasure_eq_volume_withDensity {d : ℕ} : radialAngularMeasure =
    volume.withDensity (fun x : Space d => ENNReal.ofReal (1 / ‖x‖ ^ (d - 1)))
```

lemma radialAngularMeasure_zero_eq_volume [source]

```
@[simp]
lemma radialAngularMeasure_zero_eq_volume :
    radialAngularMeasure (d := 0) = volume
```

lemma integrable_radialAngularMeasure_iff [source]

```
lemma integrable_radialAngularMeasure_iff {d : ℕ} {f : Space d → F} :
    Integrable f (radialAngularMeasure (d := d)) ↔
      Integrable (fun x => (1 / ‖x‖ ^ (d - 1)) • f x) volume
```

lemma integral_radialAngularMeasure [source]

```
lemma integral_radialAngularMeasure {d : ℕ} (f : Space d → F) :
    ∫ x, f x ∂radialAngularMeasure = ∫ x, (1 / ‖x‖ ^ (d - 1)) • f x
```

lemma integrable_neg_pow_on_ioi [source]

```
private lemma integrable_neg_pow_on_ioi (n : ℕ) :
    IntegrableOn (fun x : ℝ => (|((1 : ℝ) + ↑x) ^ (- (n + 2) : ℝ)|)) (Set.Ioi 0)
```

lemma radialAngularMeasure_integrable_pow_neg_two [source]

```
lemma radialAngularMeasure_integrable_pow_neg_two {d : ℕ} :
    Integrable (fun x : Space d => (1 + ‖x‖) ^ (- (d + 1) : ℝ))
      radialAngularMeasure
```

instance Measure.HasTemperateGrowth (radialAngularMeasure (d [source]

```
instance (d : ℕ) : Measure.HasTemperateGrowth (radialAngularMeasure (d := d)) where
```

2.36 PhysLean.SpaceAndTime.Space.Slice

PhysLean/SpaceAndTime/Space/Slice.lean on GitHub.

def slice [source]

The linear equivalence between `Space d.succ` and ℝ × `Space d` extracting the `i`th coordinate.

```
def slice {d} (i : Fin d.succ) : Space d.succ ≃L[ℝ] ℝ × Space d where
```

lemma slice_symm_apply [source]

```
lemma slice_symm_apply {d : ℕ} (i : Fin d.succ) (r : ℝ) (x : Space d) :
    (slice i).symm (r, x) = fun j =>
      Fin.insertNthEquiv (fun _ => ℝ) i (r, x) j
```

lemma slice_symm_apply_self [source]

```
@[simp]
lemma slice_symm_apply_self {d : ℕ} (i : Fin d.succ) (r : ℝ) (x : Space d) :
    (slice i).symm (r, x) i = r
```

lemma slice_symm_apply_succAbove [source]

```
@[simp]
lemma slice_symm_apply_succAbove {d : ℕ} (i : Fin d.succ) (r : ℝ) (x : Space d) (j : Fin d) :
    (slice i).symm (r, x) (Fin.succAbove i j) = x j
```

lemma slice_symm_measurableEmbedding [source]

```
lemma slice_symm_measurableEmbedding {d : ℕ} (i : Fin d.succ) :
    MeasurableEmbedding (slice i).symm
```

lemma norm_slice_symm_eq [source]

```
lemma norm_slice_symm_eq {d : ℕ} (i : Fin d.succ) (r : ℝ) (x : Space d) :
    ‖(slice i).symm (r, x)‖ = √(‖r‖ ^ 2 + ‖x‖ ^ 2)
```

lemma abs_right_le_norm_slice_symm [source]

```
lemma abs_right_le_norm_slice_symm {d : ℕ} (i : Fin d.succ) (r : ℝ) (x : Space d) :
    |r| ≤ ‖(slice i).symm (r, x)‖
```

lemma norm_left_le_norm_slice_symm [source]

```
@[simp]
lemma norm_left_le_norm_slice_symm {d : ℕ} (i : Fin d.succ) (r : ℝ) (x : Space d) :
    ‖x‖ ≤ ‖(slice i).symm (r, x)‖
```

lemma fderiv_slice_symm [source]

```
@[simp]
lemma fderiv_slice_symm {d : ℕ} (i : Fin d.succ) (p1 : ℝ × Space d) :
    fderiv ℝ (slice i).symm p1 = (slice i).symm
```

lemma fderiv_slice_symm_left_apply [source]

```
lemma fderiv_slice_symm_left_apply {d : ℕ} (i : Fin d.succ) (x : Space d) (r1 r2 : ℝ) :
    (fderiv ℝ (fun r => (slice i).symm (r, x))) r1 r2 = (slice i).symm (r2, 0)
```

lemma fderiv_slice_symm_right_apply [source]

```
@[simp]
lemma fderiv_slice_symm_right_apply {d : ℕ} (i : Fin d.succ) (r : ℝ)
    (x1 x2 : Space d) :
    (fderiv ℝ (fun x => (slice i).symm (r, x))) x1 x2 = (slice i).symm (0, x2)
```

lemma fderiv_fun_slice_symm_right_apply [source]

```
lemma fderiv_fun_slice_symm_right_apply {d : ℕ} (i : Fin d.succ) (r : ℝ)
    (x1 x2 : Space d) (f : Space d.succ → F) (hf : DifferentiableAt ℝ f ((slice i).symm (r, x1)
    )) :
    (fderiv ℝ (fun x => f ((slice i).symm (r, x)))) x1 x2 =
    fderiv ℝ f ((slice i).symm (r, x1)) ((slice i).symm (0, x2))
```

lemma fderiv_fun_slice_symm_left_apply [source]

```
lemma fderiv_fun_slice_symm_left_apply {d : ℕ} (i : Fin d.succ) (r1 r2 : ℝ)
    (x : Space d) (f : Space d.succ → F) (hf : DifferentiableAt ℝ f ((slice i).symm (r1, x))) :
    (fderiv ℝ (fun r => f ((slice i).symm (r, x)))) r1 r2 =
    fderiv ℝ f ((slice i).symm (r1, x)) ((slice i).symm (r2, 0))
```

lemma basis_self_eq_slice [source]

```
lemma basis_self_eq_slice {d : ℕ} (i : Fin d.succ) :
    basis i = (slice i).symm (1, 0)
```

lemma basis_succAbove_eq_slice [source]

```
lemma basis_succAbove_eq_slice {d : ℕ} (i : Fin d.succ) (j : Fin d) :
    basis (Fin.succAbove i j) = (slice i).symm (0, basis j)
```

2.37 PhysLean.SpaceAndTime.Space.Translations

PhysLean/SpaceAndTime/Space/Translations.lean on GitHub.

lemma translateD_distGrad [source]

```
@[simp]
lemma translateD_distGrad {d : ℕ} (a : EuclideanSpace ℝ (Fin d))
    (T : (Space d) →d[ℝ] ℝ) :
    distGrad (translateD a T) = translateD a (distGrad T)
```

lemma distDiv_translateD [source]

```
@[simp]
lemma distDiv_translateD {d : ℕ} (a : EuclideanSpace ℝ (Fin d))
    (T : (Space d) →d[ℝ] EuclideanSpace ℝ (Fin d)) :
    distDiv (translateD a T) = translateD a (distDiv T)
```

2.38 PhysLean.SpaceAndTime.SpaceTime.Basic

PhysLean/SpaceAndTime/SpaceTime/Basic.lean on GitHub.

def timeSpaceBasis [source]

The basis of `SpaceTime` where the first component is `(c, 0, 0, ...)` instead of `(1, 0, 0, ....)`.

```
def timeSpaceBasis {d : ℕ} (c : SpeedOfLight := 1) :
    Module.Basis (Fin 1 ⊕ Fin d) ℝ (SpaceTime d) where
```

lemma timeSpaceBasis_apply_inl [source]

```
@[simp]
lemma timeSpaceBasis_apply_inl {d : ℕ} (c : SpeedOfLight) :
    timeSpaceBasis (d := d) c (Sum.inl 0) = c.val • Lorentz.Vector.basis (Sum.inl 0)
```

lemma timeSpaceBasis_apply_inr [source]

```
@[simp]
lemma timeSpaceBasis_apply_inr {d : ℕ} (c : SpeedOfLight) (i : Fin d) :
    timeSpaceBasis (d := d) c (Sum.inr i) = Lorentz.Vector.basis (Sum.inr i)
```

def timeSpaceBasisEquiv [source]

The equivalence on of `SpaceTime` taking `(1, 0, 0, ...)` to of `(c, 0, 0, ....)` and keeping all other components the same.

```
def timeSpaceBasisEquiv {d : ℕ} (c : SpeedOfLight) :
    SpaceTime d ≃L[ℝ] SpaceTime d where
```

lemma det_timeSpaceBasisEquiv [source]

```
lemma det_timeSpaceBasisEquiv {d : ℕ} (c : SpeedOfLight) :
    (timeSpaceBasisEquiv (d := d) c).det = c.val
```

lemma timeSpaceBasis_eq_map_basis [source]

```
lemma timeSpaceBasis_eq_map_basis {d : ℕ} (c : SpeedOfLight) :
    timeSpaceBasis (d := d) c =
    Module.Basis.map (Lorentz.Vector.basis (d := d)) (timeSpaceBasisEquiv c).toLinearEquiv
```

lemma timeSpaceBasis_addHaar [source]

```
lemma timeSpaceBasis_addHaar {d : ℕ} (c : SpeedOfLight := 1) :
    (timeSpaceBasis (d := d) c).addHaar = (ENNReal.ofReal (c⁻¹)) • volume
```

lemma toTimeAndSpace_symm_measurePreserving [source]

```
lemma toTimeAndSpace_symm_measurePreserving {d : ℕ} (c : SpeedOfLight) :
    MeasurePreserving (toTimeAndSpace c).symm (volume.prod (volume (α := Space d)))
    (ENNReal.ofReal c⁻¹ • volume)
```

lemma spaceTime_integral_eq_time_space_integral [source]

```
lemma spaceTime_integral_eq_time_space_integral {M} [NormedAddCommGroup M]
    [NormedSpace ℝ M] {d : ℕ} (c : SpeedOfLight)
    (f : SpaceTime d → M) :
    ∫ x : SpaceTime d, f x ∂(volume) =
    c.val • ∫ tx : Time × Space d, f ((toTimeAndSpace c).symm tx) ∂(volume.prod volume)
```

lemma spaceTime_integrable_iff_space_time_integrable [source]

```
lemma spaceTime_integrable_iff_space_time_integrable {M} [NormedAddCommGroup M]
    {d : ℕ} (c : SpeedOfLight)
    (f : SpaceTime d → M) :
    Integrable f volume ↔ Integrable (f ∘ ((toTimeAndSpace c).symm)) (volume.prod volume)
```

lemma spaceTime_integral_eq_time_integral_space_integral [source]

```
lemma spaceTime_integral_eq_time_integral_space_integral {M} [NormedAddCommGroup M]
    [NormedSpace ℝ M] {d : ℕ} (c : SpeedOfLight)
    (f : SpaceTime d → M)
    (h : Integrable f volume) :
    ∫ x : SpaceTime d, f x =
    c.val • ∫ t : Time, ∫ x : Space d, f ((toTimeAndSpace c).symm (t, x))
```

lemma spaceTime_integral_eq_space_integral_time_integral [source]

```
lemma spaceTime_integral_eq_space_integral_time_integral {M} [NormedAddCommGroup M]
    [NormedSpace ℝ M] {d : ℕ} (c : SpeedOfLight)
    (f : SpaceTime d → M)
    (h : Integrable f volume) :
    ∫ x : SpaceTime d, f x =
```

```
    c.val • ∫ x : Space d, ∫ t : Time, f ((toTimeAndSpace c).symm (t, x))
```

2.39 PhysLean.SpaceAndTime.SpaceTime.Derivatives

PhysLean/SpaceAndTime/SpaceTime/Derivatives.lean on GitHub.

lemma differentiable_vector [source]

```
lemma differentiable_vector {d : ℕ} (f : SpaceTime d → Lorentz.Vector d) :
    (∀ ν, Differentiable ℝ (fun x => f x ν)) ↔ Differentiable ℝ f
```

lemma contDiff_vector [source]

```
lemma contDiff_vector {d : ℕ} (f : SpaceTime d → Lorentz.Vector d) :
    (∀ ν, ContDiff ℝ n (fun x => f x ν)) ↔ ContDiff ℝ n f
```

lemma fderiv_vector [source]

```
lemma fderiv_vector {d : ℕ} (f : SpaceTime d → Lorentz.Vector d)
    (hf : Differentiable ℝ f) (y dt : SpaceTime d) (ν : Fin 1 ⊕ Fin d) :
    fderiv ℝ f y dt ν = fderiv ℝ (fun x => f x ν) y dt
```

def distDeriv [source]

Given a distribution (function) `f : Space d →d[ℝ] M` the derivative of `f` in direction `μ`.

```
noncomputable def distDeriv {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (μ : Fin 1 ⊕ Fin d) : ((SpaceTime d) →d[ℝ] M) →ₗ[ℝ] (SpaceTime d) →d[ℝ] M where
```

lemma distDeriv_apply [source]

```
lemma distDeriv_apply {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (μ : Fin 1 ⊕ Fin d) (f : (SpaceTime d) →d[ℝ] M) (ε : 𝓢(SpaceTime d, ℝ)) :
    distDeriv μ f ε = fderivD ℝ f ε (Lorentz.Vector.basis μ)
```

lemma distDeriv_commute [source]

```
lemma distDeriv_commute {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (μ ν : Fin 1 ⊕ Fin d) (f : (SpaceTime d) →d[ℝ] M) :
    distDeriv μ (distDeriv ν f) = distDeriv ν (distDeriv μ f)
```

lemma sum_apply [source]

```
lemma _root_.SchwartzMap.sum_apply {α : Type} [NormedAddCommGroup α]
    [NormedSpace ℝ α]
    {ι : Type} [Fintype ι]
    (f : ι → 𝓢(α, ℝ)) (x : α) :
    (∑ i, f i) x = ∑ i, f i x
```

lemma distDeriv_comp_lorentz_action [source]

```
lemma distDeriv_comp_lorentz_action {μ : Fin 1 ⊕ Fin d} (Λ : LorentzGroup d)
    (f : (SpaceTime d) →d[ℝ] M) :
    distDeriv μ (Λ • f) = ∑ ν, Λ⁻¹.1 ν μ • (Λ • distDeriv ν f)
```

2.40 PhysLean.SpaceAndTime.SpaceTime.LorentzAction

PhysLean/SpaceAndTime/SpaceTime/LorentzAction.lean on GitHub.

def schwartzAction [source]

The Lorentz group action on Schwartz functions taking the Lorentz group to continous linear maps.

```
def schwartzAction {d} : LorentzGroup d →* 𝓢(SpaceTime d, ℝ) →L[ℝ] 𝓢(SpaceTime d, ℝ) where
```

lemma schwartzAction_mul_apply [source]

```
lemma schwartzAction_mul_apply {d} (Λ₁ Λ₂ : LorentzGroup d)
    (η : 𝓢(SpaceTime d, ℝ)) :
    schwartzAction Λ₂ (schwartzAction (Λ₁) η) =
    schwartzAction (Λ₂ * Λ₁) η
```

lemma schwartzAction_apply [source]

```
lemma schwartzAction_apply {d} (Λ : LorentzGroup d)
    (η : 𝓢(SpaceTime d, ℝ)) (x : SpaceTime d) :
    (schwartzAction Λ η) x = η (Λ⁻¹ • x)
```

lemma schwartzAction_injective [source]

```
lemma schwartzAction_injective {d} (Λ : LorentzGroup d) :
    Function.Injective (schwartzAction Λ)
```

lemma schwartzAction_surjective [source]

```
lemma schwartzAction_surjective {d} (Λ : LorentzGroup d) :
    Function.Surjective (schwartzAction Λ)
```

instance SMul (LorentzGroup d) ((SpaceTime d) →d[ℝ] M) [source]

```
instance : SMul (LorentzGroup d) ((SpaceTime d) →d[ℝ] M) where
```

lemma lorentzGroup_smul_dist_apply [source]

```
lemma lorentzGroup_smul_dist_apply (Λ : LorentzGroup d) (f : (SpaceTime d) →d[ℝ] M)
    (η : 𝓢(SpaceTime d, ℝ)) : (Λ • f) η = Λ • (f (schwartzAction Λ⁻¹ η))
```

instance DistribMulAction (LorentzGroup d) ((SpaceTime d) →d[ℝ] M) [source]

```
instance : DistribMulAction (LorentzGroup d) ((SpaceTime d) →d[ℝ] M) where
```

instance SMulCommClass ℝ (LorentzGroup d) ((SpaceTime d) →d[ℝ] M) [source]

```
instance : SMulCommClass ℝ (LorentzGroup d) ((SpaceTime d) →d[ℝ] M) where
```

2.41 PhysLean.SpaceAndTime.SpaceTime.TimeSlice

PhysLean/SpaceAndTime/SpaceTime/TimeSlice.lean on GitHub.

def distTimeSlice [source]

The time slice of a distribution on `SpaceTime d` to form a distribution on `Time × Space d`.

```
noncomputable def distTimeSlice {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (c : SpeedOfLight := 1) :
    ((SpaceTime d) →d[ℝ] M) ≃L[ℝ] ((Time × Space d) →d[ℝ] M) where
```

lemma distTimeSlice_apply [source]

```
lemma distTimeSlice_apply {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (c : SpeedOfLight) (f : (SpaceTime d) →d[ℝ] M)
    (κ : 𝓢(Time × Space d, ℝ)) : distTimeSlice c f κ =
    f (compCLMOfContinuousLinearEquiv ℝ (toTimeAndSpace c) κ)
```

lemma distTimeSlice_symm_apply [source]

```
lemma distTimeSlice_symm_apply {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (c : SpeedOfLight) (f : (Time × (Space d)) →d[ℝ] M)
    (κ : 𝓢(SpaceTime d, ℝ)) : (distTimeSlice c).symm f κ =
    f (compCLMOfContinuousLinearEquiv ℝ (toTimeAndSpace c).symm κ)
```

lemma distTimeSlice_distDeriv_inl [source]

```
lemma distTimeSlice_distDeriv_inl {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    {c : SpeedOfLight}
    (f : (SpaceTime d) →d[ℝ] M) :
    distTimeSlice c (distDeriv (Sum.inl 0) f) =
    (1/c.val) • Space.distTimeDeriv (distTimeSlice c f)
```

lemma distDeriv_inl_distTimeSlice_symm [source]

```
lemma distDeriv_inl_distTimeSlice_symm {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    {c : SpeedOfLight}
    (f : (Time × Space d) →d[ℝ] M) :
    distDeriv (Sum.inl 0) ((distTimeSlice c).symm f) =
    (1/c.val) • (distTimeSlice c).symm (Space.distTimeDeriv f)
```

lemma distTimeSlice_distDeriv_inr [source]

```
lemma distTimeSlice_distDeriv_inr {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    {c : SpeedOfLight}
    (i : Fin d) (f : (SpaceTime d) →d[ℝ] M) :
    distTimeSlice c (distDeriv (Sum.inr i) f) =
    Space.distSpaceDeriv i (distTimeSlice c f)
```

lemma distDeriv_inr_distTimeSlice_symm [source]

```
lemma distDeriv_inr_distTimeSlice_symm {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    {c : SpeedOfLight}
    (i : Fin d) (f : (Time × Space d) →d[ℝ] M) :
    distDeriv (Sum.inr i) ((distTimeSlice c).symm f) =
    (distTimeSlice c).symm (Space.distSpaceDeriv i f)
```

2.42 PhysLean.SpaceAndTime.Time.Basic

PhysLean/SpaceAndTime/Time/Basic.lean on GitHub.

def basis [source]

The orthonomral basis on `Time` defined by `1`.

```
noncomputable def basis : OrthonormalBasis (Fin 1) ℝ Time where
```

lemma basis_apply_eq_one [source]

```
@[simp]
lemma basis_apply_eq_one (i : Fin 1) :
    basis i = 1
```

lemma rank_eq_one [source]

```
@[simp]
lemma rank_eq_one : Module.rank ℝ Time = 1
```

lemma finRank_eq_one [source]

```
@[simp]
lemma finRank_eq_one : Module.finrank ℝ Time = 1
```

lemma volume_eq_basis_addHaar [source]

```
lemma volume_eq_basis_addHaar :
    (volume (α := Time)) = basis.toBasis.addHaar
```

2.43 PhysLean.SpaceAndTime.Time.Derivatives

PhysLean/SpaceAndTime/Time/Derivatives.lean on GitHub.

lemma differentiable_euclid [source]

```
lemma differentiable_euclid {f : Time → EuclideanSpace ℝ (Fin n)}
    (hf : ∀ i, Differentiable ℝ (fun t => f t i)) :
    Differentiable ℝ f
```

lemma fderiv_euclid [source]

```
lemma fderiv_euclid { μ} {f : Time→ EuclideanSpace ℝ (Fin n)}
    (hf : Differentiable ℝ f) (t dt : Time) :
    fderiv ℝ (fun t => f t μ) t dt = fderiv ℝ (fun t => f t) t dt μ
```

2.44 PhysLean.SpaceAndTime.TimeAndSpace.Basic

PhysLean/SpaceAndTime/TimeAndSpace/Basic.lean on GitHub.

lemma fderiv_space_eq_fderiv_curry [source]

```
lemma fderiv_space_eq_fderiv_curry {M} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (f : Time → Space d → M) (t : Time) (x dx : Space d)
    (hf : Differentiable ℝ ↿f) :
    fderiv ℝ (fun x' => f t x') x dx = fderiv ℝ ↿f (t, x) (0, dx)
```

lemma fderiv_time_eq_fderiv_curry [source]

```
lemma fderiv_time_eq_fderiv_curry {M} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (f : Time → Space d → M) (t dt : Time) (x : Space d)
    (hf : Differentiable ℝ ↿f) :
    fderiv ℝ (fun t' => f t' x) t dt = fderiv ℝ ↿f (t, x) (dt, 0)
```

lemma fderiv_time_commute_fderiv_space [source]

Derivatives along space coordinates and time commute.

```
lemma fderiv_time_commute_fderiv_space {M} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (f : Time → Space d → M) (t dt : Time) (x dx : Space d)
    (hf : ContDiff ℝ 2 ↿f) :
    fderiv ℝ (fun t' => fderiv ℝ (fun x' => f t' x') x dx) t dt
    = fderiv ℝ (fun x' => fderiv ℝ (fun t' => f t' x') t dt) x dx
```

lemma time_deriv_comm_space_deriv [source]

```
lemma time_deriv_comm_space_deriv {d i} {M} [NormedAddCommGroup M] [NormedSpace ℝ M]
    {f : Time → Space d → M} (hf : ContDiff ℝ 2 ↿f) (t : Time) (x : Space d) :
    Time.deriv (fun t' => Space.deriv i (f t') x) t
    = Space.deriv i (fun x' => Time.deriv (fun t' => f t' x') t) x
```

lemma space_deriv_differentiable_time [source]

```
@[fun_prop]
lemma space_deriv_differentiable_time {d i} {M} [NormedAddCommGroup M] [NormedSpace ℝ M]
    {f : Time → Space d → M} (hf : ContDiff ℝ 2 ↿f) (x : Space d) :
    Differentiable ℝ (fun t => Space.deriv i (f t) x)
```

lemma time_deriv_differentiable_space [source]

```
@[fun_prop]
lemma time_deriv_differentiable_space {d } {M} [NormedAddCommGroup M] [NormedSpace ℝ M]
    {f : Time → Space d → M} (hf : ContDiff ℝ 2 ↿f) (t : Time) :
    Differentiable ℝ (fun x => Time.deriv (f · x) t)
```

lemma curl_differentiable_time [source]

```
@[fun_prop]
lemma curl_differentiable_time
    (fₜ : Time → Space → EuclideanSpace ℝ (Fin 3))
    (hf : ContDiff ℝ 2 ₁fₜ) (x : Space) :
    Differentiable ℝ (fun t => (∇ × fₜ t) x)
```

lemma time_deriv_curl_commute [source]

Curl and time derivative commute.

```
lemma time_deriv_curl_commute (fₜ : Time → Space → EuclideanSpace ℝ (Fin 3))
    (t : Time) (x : Space) (hf : ContDiff ℝ 2 ₁fₜ) :
    ∂ₜ (fun t => (∇ × fₜ t) x) t = (∇ × fun x => (∂ₜ (fun t => fₜ t x) t)) x
```

lemma space_fun_of_time_deriv_eq_zero [source]

```
lemma space_fun_of_time_deriv_eq_zero {d} {M} [NormedAddCommGroup M] [NormedSpace ℝ M]
    {f : Time → Space d → M} (hf : Differentiable ℝ ₁f)
    (h : ∀ t x, ∂ₜ (f · x) t = 0) :
    ∃ (g : Space d → M), ∀ t x, f t x = g x
```

lemma time_fun_of_space_deriv_eq_zero [source]

```
lemma time_fun_of_space_deriv_eq_zero {d} {M} [NormedAddCommGroup M] [NormedSpace ℝ M]
    {f : Time → Space d → M} (hf : Differentiable ℝ ₁f)
    (h : ∀ t x i, Space.deriv i (f t) x = 0) :
    ∃ (g : Time → M), ∀ t x, f t x = g t
```

lemma const_of_time_deriv_space_deriv_eq_zero [source]

```
lemma const_of_time_deriv_space_deriv_eq_zero {d} {M} [NormedAddCommGroup M] [NormedSpace ℝ M]
    {f : Time → Space d → M} (hf : Differentiable ℝ ₁f)
    (h₁ : ∀ t x, ∂ₜ (f · x) t = 0)
    (h₂ : ∀ t x i, Space.deriv i (f t) x = 0) :
    ∃ (c : M), ∀ t x, f t x = c
```

lemma equal_up_to_const_of_deriv_eq [source]

```
lemma equal_up_to_const_of_deriv_eq {d} {M} [NormedAddCommGroup M] [NormedSpace ℝ M]
    {f g : Time → Space d → M} (hf : Differentiable ℝ ₁f) (hg : Differentiable ℝ ₁g)
    (h₁ : ∀ t x, ∂ₜ (f · x) t = ∂ₜ (g · x) t)
    (h₂ : ∀ t x i, Space.deriv i (f t) x = Space.deriv i (g t) x) :
    ∃ (c : M), ∀ t x, f t x = g t x + c
```

def distTimeDeriv [source]

The time derivative of a distribution dependent on time and space.

```
noncomputable def distTimeDeriv {M d} [NormedAddCommGroup M] [NormedSpace ℝ M] :
    ((Time × Space d) →d[ℝ] M) →ₗ[ℝ] (Time × Space d) →d[ℝ] M where
```

lemma distTimeDeriv_apply [source]

```
lemma distTimeDeriv_apply {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (f : (Time × Space d) →d[ℝ] M) (ε : 𝓢(Time × Space d, ℝ)) :
    (distTimeDeriv f) ε = fderivD ℝ f ε (1, 0)
```

lemma distTimeDeriv_apply_CLM [source]

```
lemma distTimeDeriv_apply_CLM {M M2 d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    [NormedAddCommGroup M2] [NormedSpace ℝ M2] (f : (Time × Space d) →d[ℝ] M)
    (c : M →L[ℝ] M2) : distTimeDeriv (c ∘L f) = c ∘L (distTimeDeriv f)
```

def distSpaceDeriv [source]

The space derivative of a distribution dependent on time and space.

```
noncomputable def distSpaceDeriv {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (i : Fin d) : ((Time × Space d) →d[ℝ] M) →ₗ[ℝ] (Time × Space d) →d[ℝ] M where
```

lemma distSpaceDeriv_apply [source]

```
lemma distSpaceDeriv_apply {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (i : Fin d) (f : (Time × Space d) →d[ℝ] M) (ε : 𝓢(Time × Space d, ℝ)) :
    (distSpaceDeriv i f) ε = fderivD ℝ f ε (0, basis i)
```

lemma distSpaceDeriv_commute [source]

```
lemma distSpaceDeriv_commute {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (i j : Fin d) (f : (Time × Space d) →d[ℝ] M) :
    distSpaceDeriv i (distSpaceDeriv j f) = distSpaceDeriv j (distSpaceDeriv i f)
```

lemma distSpaceDeriv_apply_CLM [source]

```
lemma distSpaceDeriv_apply_CLM {M M2 d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    [NormedAddCommGroup M2] [NormedSpace ℝ M2]
    (i : Fin d) (f : (Time × Space d) →d[ℝ] M)
    (c : M →L[ℝ] M2) : distSpaceDeriv i (c ∘L f) = c ∘L (distSpaceDeriv i f)
```

lemma distTimeDeriv_commute_distSpaceDeriv [source]

```
lemma distTimeDeriv_commute_distSpaceDeriv {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (i : Fin d) (f : (Time × Space d) →d[ℝ] M) :
    distTimeDeriv (distSpaceDeriv i f) = distSpaceDeriv i (distTimeDeriv f)
```

def distSpaceGrad [source]

The spatial gradient of a distribution dependent on time and space.

```
noncomputable def distSpaceGrad {d} :
    ((Time × Space d) →d[ℝ] ℝ) →ₗ[ℝ] (Time × Space d) →d[ℝ] (EuclideanSpace ℝ (Fin d)) where
```

lemma distSpaceGrad_apply [source]

```
lemma distSpaceGrad_apply {d} (f : (Time × Space d) →d[ℝ] ℝ) (ε : 𝓢(Time × Space d, ℝ)) :
    distSpaceGrad f ε = fun i => distSpaceDeriv i f ε
```

def distSpaceDiv [source]

The spatial divergence of a distribution dependent on time and space.

```
noncomputable def distSpaceDiv {d} :
    ((Time × Space d) →d[ℝ] (EuclideanSpace ℝ (Fin d))) →ₗ[ℝ] (Time × Space d) →d[ℝ] ℝ where
```

lemma distSpaceDiv_apply_eq_sum_distSpaceDeriv [source]

```
lemma distSpaceDiv_apply_eq_sum_distSpaceDeriv {d}
    (f : (Time × Space d) →d[ℝ] EuclideanSpace ℝ (Fin d)) (η : 𝓢(Time ×Space d, ℝ)) :
    distSpaceDiv f η = ∑ i, distSpaceDeriv i f η i
```

def distSpaceCurl [source]

The curl of a distribution dependent on time and space.

```
noncomputable def distSpaceCurl : ((Time × Space 3) →d[ℝ] (EuclideanSpace ℝ (Fin 3))) →ₗ[ℝ]
    (Time × Space 3) →d[ℝ] (EuclideanSpace ℝ (Fin 3)) where
```

2.45 PhysLean.SpaceAndTime.TimeAndSpace.ConstantTimeDist

PhysLean/SpaceAndTime/TimeAndSpace/ConstantTimeDist.lean on GitHub.

lemma constantTime_distSpaceDeriv [source]

```
lemma constantTime_distSpaceDeriv {M : Type} {d : ℕ} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (i : Fin d) (f : (Space d) →d[ℝ] M) :
    Space.distSpaceDeriv i (constantTime f) = constantTime (Space.distDeriv i f)
```

lemma constantTime_distSpaceGrad [source]

```
lemma constantTime_distSpaceGrad {d : ℕ} (f : (Space d) →d[ℝ] ℝ) :
    Space.distSpaceGrad (constantTime f) = constantTime (Space.distGrad f)
```

lemma constantTime_distSpaceDiv [source]

```
lemma constantTime_distSpaceDiv {d : ℕ} (f : (Space d) →d[ℝ] EuclideanSpace ℝ (Fin d)) :
    Space.distSpaceDiv (constantTime f) = constantTime (Space.distDiv f)
```

lemma constantTime_distTimeDeriv [source]

```
@[simp]
lemma constantTime_distTimeDeriv {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M] {d : ℕ}
    (f : (Space d) →d[ℝ] M) :
    Space.distTimeDeriv (constantTime f) = 0
```